LAPORAN AKHIR
PRAKTIKUM
PEMROGRAMAN MOBILE

Oleh:

Bi'ahlil Haq Aulia Akbar Awaludin NIM. 2010817110011

PROGRAM STUDI TEKNOLOGI INFORMASI

FAKULTAS TEKNIK

UNIVERSITAS LAMBUNG MANGKURAT

JUNI 2025

LEMBAR PENGESAHAN

LAPORAN AKHIR

PRAKTIKUM PEMROGRAMAN II

Laporan Akhir Praktikum Pemrograman Mobile ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Bi'ahlil Haq Aulia Akbar Awaludin

NIM : 2010817110011

Menyetujui,	Mengetahui,
Asisten Praktikum	Dosen Penanggung Jawab Praktikum
Muhammad Raka Azwar	Ir. EKA SETYA WIJAYA, S.T., M.Kom
NIM. 2210817210012	NIP. 19820508 200801 10 10

DAFTAR ISI

DAFTAR GAMBAR

DAFTAR TABEL

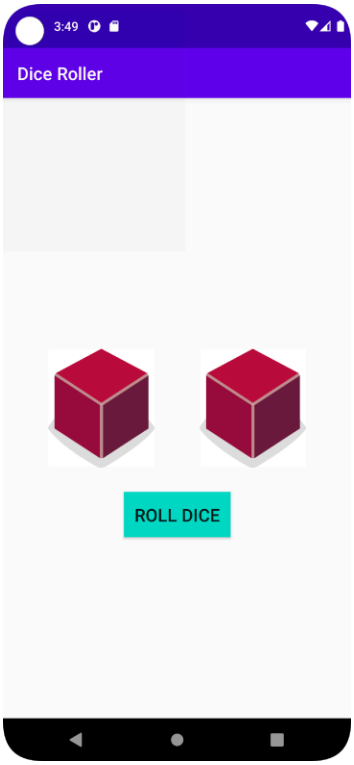
MODUL 1

SOAL

Soal Praktikum:

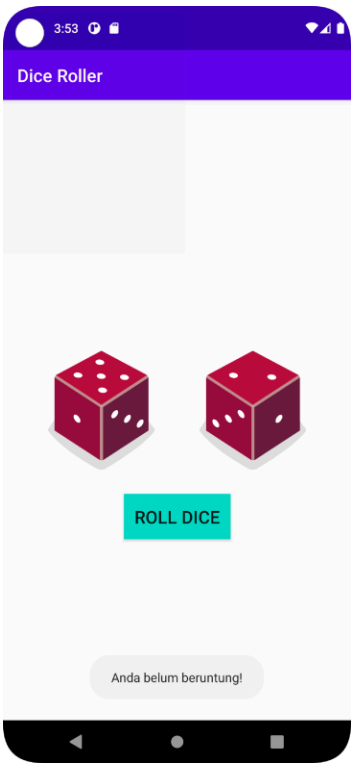
Buatlah sebuah aplikasi yang dapat menampilkan 2 (dua) buah dadu yang dapat berubah-ubah tampilannya pada saat user menekan tombol “Roll Dice”. Aturan aplikasi yang akan dibangun adalah sebagaimana berikut:

1. Tampilan awal aplikasi setelah dijalankan akan menampilkan 2 buah dadu kosong seperti dapat dilihat pada Gambar 1.



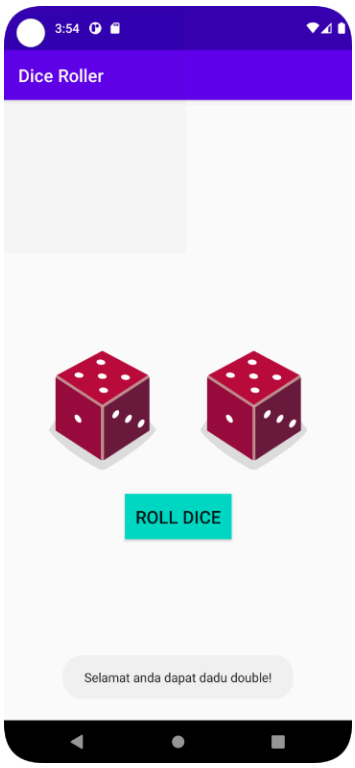
Gambar 1. Tampilan Awal Aplikasi

2. Setelah user menekan tombol “Roll” maka masing-masing dadu akan memperlihatkan sisi dadunya dengan angka antara 1 s/d 6. Apabila user mendapatkan nilai dadu yang berbeda antara Dadu 1 dengan Dadu 2, maka aplikasi akan menampilkan pesan “Anda belum beruntung!” seperti yang dapat dilihat pada Gambar 2.



Gambar 2. Tampilan Dadu Setelah Di Roll

3. Apabila user mendapatkan nilai dadu yang sama antara Dadu 1 dan Dadu 2 atau nilai double, maka aplikasi akan menampilkan pesan “Selamat, anda dapat dadu double!” seperti dapat dilihat pada Gambar 3.



Gambar 3. Tampilan Roll Dadu Double

4. Buatlah aplikasi tersebut menggunakan XML dan Jetpack Compose.

5. Upload aplikasi yang telah anda buat ke dalam repository GitHub ke dalam **folder Modul 1 dalam bentuk Project.** Jangan lupa untuk melakukan **Clean Project** sebelum mengupload pekerjaan anda pada repository.

   Untuk gambar dadu dapat didownload pada link berikut: https://drive.google.com/file/d/14V3qXGdFnuoYN4AGd_9SgFh8kw8X9ySm/view?usp= sharing

A. Source Code

Compose

MainActivity.kt

```
1   package com.example.diceroller
2
3   import android.os.Bundle
4   import androidx.activity.ComponentActivity
5   import androidx.activity.compose.setContent
6   import androidx.activity.enableEdgeToEdge
7   import androidx.compose.foundation.Image
8   import androidx.compose.foundation.background
9   import androidx.compose.foundation.layout.Arrangement
10  import androidx.compose.foundation.layout.Column
11  import androidx.compose.foundation.layout.Row
12  import androidx.compose.foundation.layout.Spacer
13  import androidx.compose.foundation.layout.fillMaxSize
14  import androidx.compose.foundation.layout.fillMaxWidth
15  import androidx.compose.foundation.layout.height
16  import androidx.compose.foundation.layout.padding
17  import androidx.compose.foundation.layout.wrapContentSize
18  import androidx.compose.material3.BottomAppBar
19  import androidx.compose.material3.Button
20  import androidx.compose.material3.MaterialTheme
21  import androidx.compose.material3.Scaffold
22  import androidx.compose.material3.Surface
23  import androidx.compose.material3.Text
24  import androidx.compose.runtime.Composable
25  import androidx.compose.runtime.getValue
26  import androidx.compose.runtime.mutableIntStateOf
27  import androidx.compose.runtime.mutableStateOf
28  import androidx.compose.runtime.remember
29  import androidx.compose.runtime.setValue
```

```
30  import androidx.compose.ui.Alignment
31  import androidx.compose.ui.Modifier
32  import androidx.compose.ui.graphics.Color
33  import androidx.compose.ui.res.painterResource
34  import androidx.compose.ui.res.stringResource
35  import androidx.compose.ui.tooling.preview.Preview
36  import androidx.compose.ui.unit.dp
37  import com.example.diceroller.ui.theme.DiceRollerTheme
38
39  class MainActivity : ComponentActivity() {
40      override fun onCreate(savedInstanceState: Bundle?) {
41          super.onCreate(savedInstanceState)
42          enableEdgeToEdge()
43          setContent {
44              DiceRollerTheme {
45                  Surface(
46                      modifier = Modifier.fillMaxSize(),
47                  ) {
48                      DiceRollerApp()
49                  }
50              }
51          }
52      }
53  }
54
55  @Preview
56  @Composable
57  fun DiceRollerApp() {
58      DiceWithButtonAndImage(modifier = Modifier
59          .fillMaxSize()
60          .wrapContentSize(Alignment.Center)
61          .background(MaterialTheme.colorScheme.background)
62      )
63  }
64
```

```
@Composable
private fun DiceWithButtonAndImage(modifier: Modifier = Modifier) {
    var dice1 by remember { mutableIntStateOf(0) }
    var dice2 by remember { mutableIntStateOf(0) }
    val imageResource1 = when (dice1) {
        0 -> R.drawable.dice_0
        1 -> R.drawable.dice_1
        2 -> R.drawable.dice_2
        3 -> R.drawable.dice_3
        4 -> R.drawable.dice_4
        5 -> R.drawable.dice_5
        else -> R.drawable.dice_6
    }

    val imageResource2 = when (dice2) {
        0 -> R.drawable.dice_0
        1 -> R.drawable.dice_1
        2 -> R.drawable.dice_2
        3 -> R.drawable.dice_3
        4 -> R.drawable.dice_4
        5 -> R.drawable.dice_5
        else -> R.drawable.dice_6
    }
    Column(
        modifier = Modifier,
        horizontalAlignment = Alignment.CenterHorizontally
        ) {
        Spacer(modifier = Modifier.weight(1f))
        Row {
            Image(painter = painterResource(imageResource1), contentDescription = dice1.toString())
            Image(painter = painterResource(imageResource2), contentDescription = dice2.toString())
        }

```

```
98          Spacer(modifier = Modifier.height(22.dp))
99          Button(onClick = {
100             dice1 = (1..6).random()
101             dice2 = (1..6).random()
102             }
103         ) { Text(stringResource(R.string.roll)) }
104         Spacer(modifier = Modifier.weight(1f))
105         if (dice1 != 0 || dice2 != 0)
106             Text(
107                 stringResource(
108                      if (dice1 == dice2) R.string.twins else
R.string.zonk
109                 ),
110                                                          color   =
MaterialTheme.colorScheme.background,
111                                                          style   =
MaterialTheme.typography.labelMedium,
112                 modifier = Modifier
113                     .fillMaxWidth()
114
115 .background(MaterialTheme.colorScheme.primary)
116                     .padding(8.dp)
117             )
    }
}
```

Tabel 1. Source Code Jawaban Soal 1

XML

MainActivity.kt

```
1   package com.example.dicerollerxml
2
3   import android.os.Bundle
4   import android.widget.Button
5   import android.widget.ImageView
6   import android.widget.TextView
7   import android.widget.Toast
```

```
import androidx.activity.enableEdgeToEdge
import androidx.appcompat.app.AppCompatActivity
import androidx.core.view.ViewCompat
import androidx.core.view.WindowInsetsCompat

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContentView(R.layout.activity_main)
        val diceImage1: ImageView = findViewById(R.id.dice1)
        val diceImage2: ImageView = findViewById(R.id.dice2)
                                val rollButton: Button =
findViewById(R.id.rollButton)
                               val bottomText: TextView =
findViewById(R.id.messageText)


        rollButton.setOnClickListener {
            val dice1 = (1..6).random()
            val dice2 = (1..6).random()
            val drawable1 = when (dice1) {
                1 -> R.drawable.dice_1
                2 -> R.drawable.dice_2
                3 -> R.drawable.dice_3
                4 -> R.drawable.dice_4
                5 -> R.drawable.dice_5
                else -> R.drawable.dice_6
            }
            val drawable2 = when (dice2) {
                1 -> R.drawable.dice_1
                2 -> R.drawable.dice_2
                3 -> R.drawable.dice_3
                4 -> R.drawable.dice_4
                5 -> R.drawable.dice_5
                else -> R.drawable.dice_6

```

```
            }
            diceImage1.setImageResource(drawable1)
            diceImage2.setImageResource(drawable2)

            bottomText.apply {
                text = getString(
                     if (dice1 == dice2) R.string.twins else
R.string.zonk
                )

setBackgroundColor(getColor(R.color.purple_500))
                setTextColor(getColor(R.color.white))
                setPadding(16, 16, 16, 16)
                visibility = TextView.VISIBLE
            }
        }


ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.
main)) { v, insets ->
                                        val   systemBars   =
insets.getInsets(WindowInsetsCompat.Type.systemBars())
               v.setPadding(systemBars.left, systemBars.top,
systemBars.right, systemBars.bottom)
            insets
        }
    }
}
```

Tabel 2. Source Code Jawaban Soal 1

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"
```

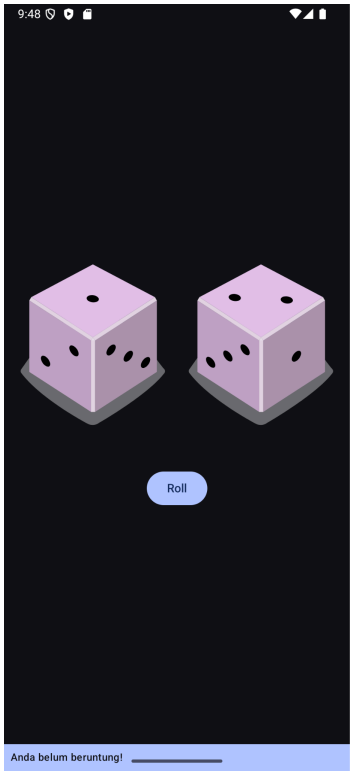
```
4       xmlns:app="http://schemas.android.com/apk/res-auto"
5       xmlns:tools="http://schemas.android.com/tools"
6       android:id="@+id/main"
7       android:layout_width="match_parent"
8       android:layout_height="match_parent"
9       android:contentDescription="@string/dice2"
10      tools:context=".MainActivity"
11      tools:layout_editor_absoluteX="10dp"
12      tools:layout_editor_absoluteY="32dp">
13
14
15      <ImageView
16          android:id="@+id/dice1"
17          android:layout_width="100dp"
18          android:layout_height="100dp"
19
20          android:layout_marginTop="260dp"
21          android:contentDescription="@string/dice1"
22          android:src="@drawable/dice_0"
23
24          app:layout_constraintEnd_toStartOf="@+id/dice2"
25          app:layout_constraintStart_toStartOf="parent"
26          app:layout_constraintHorizontal_chainStyle="packed"
27
28          app:layout_constraintTop_toTopOf="parent"
29
30  app:layout_constraintBottom_toTopOf="@+id/rollButton"
31          app:layout_constraintVertical_bias="0.0"
32          />
33
34      <ImageView
35          android:id="@+id/dice2"
36          android:layout_width="100dp"
37          android:layout_height="100dp"
38
```

```
        android:contentDescription="@string/dice2"

        android:src="@drawable/dice_0"
        app:layout_constraintBottom_toBottomOf="@+id/dice1"
app:layout_constraintBottom_toTopOf="@+id/rollButton"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintHorizontal_chainStyle="packed"

        app:layout_constraintStart_toEndOf="@+id/dice1"
        app:layout_constraintTop_toTopOf="@+id/dice1"
        app:layout_constraintVertical_bias="0.0" />

    <Button
        android:id="@+id/rollButton"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="40dp"
        android:backgroundTint="#D0BFFF"
        android:text="@string/roller"
        android:textColor="#000000"

        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintHorizontal_bias="0.498"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@id/dice1"

        />

    <TextView
        android:id="@+id/messageText"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_margin="10dp"
        android:layout_marginBottom="8dp"
```

```
73          android:background="#FFFFFF"
74          android:padding="12dp"
75          android:text="@string/zonk"
76          android:textColor="#000000"
77          android:visibility="invisible"
78          app:layout_constraintBottom_toBottomOf="parent"
79          app:layout_constraintEnd_toEndOf="parent"
80          app:layout_constraintHorizontal_bias="1.0"
            app:layout_constraintStart_toStartOf="parent" />
    </androidx.constraintlayout.widget.ConstraintLayout>
```

Tabel 3. Source Code Jawaban Soal 1

B. Output Program



Gambar 1. Screenshot Hasil Jawaban Soal 1 Compose

Gambar 2. Screenshot Hasil Jawaban Soal 1 XML

C. Pembahasan

Compose

MainActivity.kt:

Pada line 44, dideklarasikan theme default yang dibuat oleh compose secara otomatis, didalamnya akan dipanggil fungsi dicerollerapp yang membungkus keseluruhan ui

Pada line 57, dideklarasikan preview sekaligus fungsi dicerollerapp yang membungkus keseluruhan ui

Pada line 65, dideklarasikan fungsi DiceWithButtonAndImage untuk ui image dice dan buttonnya

Pada line 68 - 86, terdapat logic untuk mengatur source dari image painter sesuai dengan state nya, disini juga dideklarasikan variabel untuk menyimpan state dice1 dan dice2

Pada line 67, diimport Column layout milik compose untuk mengatur image dice dan button

Pada line 92, diimport Row layout milik compose untuk mengatur image dice 1 dan dice 2 sehingga bisa bersebelahan

Pada line 103, terdapat logic untuk menampilkan Text milik compose jika state dari dice 1 dan dice 2 berubah

Pada line 105, terdapat logic untuk mengubah stringresource Text milik compose jika value dari dice 1 dan dice 2 sama

XML

MainActivity.kt:

Pada line 7, diimport AppCompatActivity untuk base class activity yang mendukung versi terdahulu Android

Pada line 17, diimport fungsi setContentView untuk menjadikan activity_main sebagai layout tujuan

Pada line 18 - 23, dideklarasikan variable yang merujuk pada setiap komponen ui yang ada pada layout

Pada line 25, terdapat fungsi yang memuat logic apabila button di klik

Pada line 26 - 45, dideklarasikan variable untuk menyimpan nilai dadu dan logic untuk mengubah tampilan image dadu sesuai dengan nilai dari dadu

Pada line 47 - 58, dipanggil scope fungsi pada bottomText untuk memberikan logic dimana text tidak akan muncul jika button tidak diklik, dan text akan berubah jika dadu 1 dan 2 memiliki nilai sama atau tidak

activity_main.xml:

Pada line 15 - 31, dideklarasikan komponen imageview untuk dadu 1, dengan constraint endtostartof ke dadu 2 dan juga constraintHorizontal_chainStyle packed

Pada line 33 - 48, dideklarasikan komponen imageview untuk dadu 2, dengan constraint Bottom_toBottomOf ke dice1, Bottom_toTopOf ke rollButton, Horizontal_chainStyle packed

Pada line 50 - 64, dideklarasikan komponen button untuk rollButton, dengan constraint Top_toBottomOf ke dice1

Pada line 66 - 80, dideklarasikan komponen textview untuk messageText, dengan visibility invisible, dan constraint Bottom_toBottomOf ke parent, End_toEndOf ke parent, Start_toStartOf ke parent

D. Tautan Git

Berikut adalah tautan untuk source code yang telah dibuat.

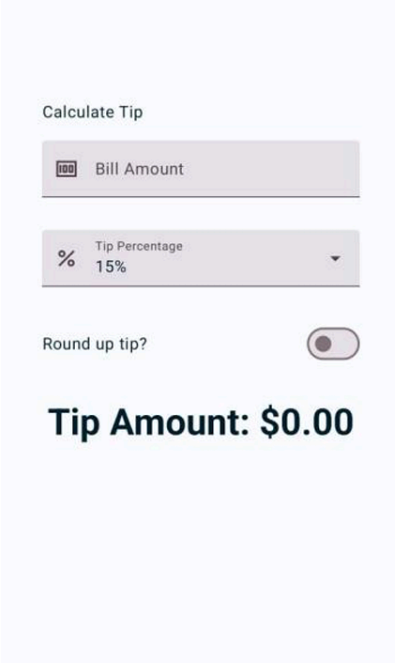
https://github.com/biahlil/Pemrograman-Mobile.git

MODUL 2

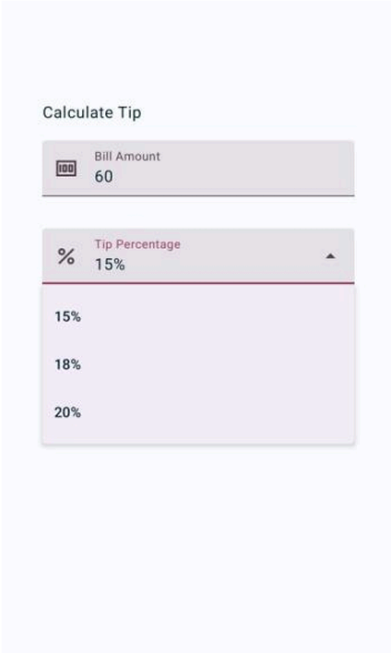
SOAL

Soal Praktikum:

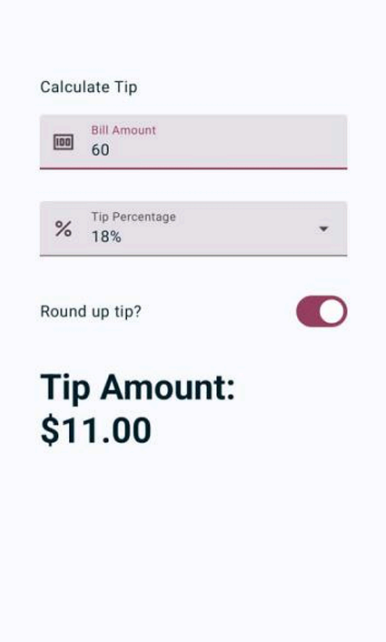
1. Buatlah sebuah aplikasi kalkulator tip menggunakan XML dan Jetpack Compose yang dirancang untuk membantu pengguna menghitung tip yang sesuai berdasarkan total biaya layanan yang mereka terima. Fitur-fitur yang diharapkan dalam aplikasi ini mencakup:
    a. Input biaya layanan: Pengguna dapat memasukkan total biaya layanan yang diterima dalam bentuk nominal.
    b. Pilihan persentase tip: Pengguna dapat memilih persentase tip yang diinginkan.
    c. Pengaturan pembulatan tip: Pengguna dapat memilih untuk membulatkan tip ke angka yang lebih tinggi.
    d. Tampilan hasil: Aplikasi akan menampilkan jumlah tip yang harus dibayar secara langsung setelah pengguna memberikan input.
2. Jelaskan perbedaan dari implementasi XML dan Jetpack Compose beserta kelebihan dan kekurangan dari masing-masing implementasi.



Gambar 4. Hasil Soal 1



Gambar 5. Hasil Soal 1



Gambar 6. Hasil Soal 1

A. Source Code

Compose
MainActivity.kt

```
1   @file:OptIn(ExperimentalMaterial3Api::class)
2   package com.example.tiptime
3   import android.os.Bundle
4   import androidx.activity.ComponentActivity
5   import androidx.activity.compose.setContent
6   import androidx.activity.enableEdgeToEdge
7   import androidx.annotation.DrawableRes
8   import androidx.annotation.StringRes
9   import androidx.compose.foundation.clickable
10  import androidx.compose.foundation.layout.Arrangement
11  import androidx.compose.foundation.layout.Box
12  import androidx.compose.foundation.layout.Column
13  import androidx.compose.foundation.layout.Row
14  import androidx.compose.foundation.layout.Spacer
15  import androidx.compose.foundation.layout.fillMaxSize
16  import androidx.compose.foundation.layout.fillMaxWidth
17  import androidx.compose.foundation.layout.height
18  import androidx.compose.foundation.layout.padding
19  import androidx.compose.foundation.layout.safeDrawingPadding
20  import androidx.compose.foundation.layout.size
21  import androidx.compose.foundation.layout.statusBarsPadding
22  import androidx.compose.foundation.rememberScrollState
23  import androidx.compose.foundation.text.KeyboardOptions
24  import androidx.compose.foundation.verticalScroll
25  import androidx.compose.material.icons.Icons
26  import
27  androidx.compose.material.icons.outlined.ArrowDropDown
28  import androidx.compose.material3.DropdownMenu
29  import androidx.compose.material3.DropdownMenuItem
30  import androidx.compose.material3.ExperimentalMaterial3Api
31  import androidx.compose.material3.Icon
32  import androidx.compose.material3.MaterialTheme
33  import androidx.compose.material3.Surface
34  import androidx.compose.material3.Switch
35  import androidx.compose.material3.Text
36  import androidx.compose.material3.TextField
37  import androidx.compose.runtime.Composable
38  import androidx.compose.runtime.getValue
39  import androidx.compose.runtime.mutableStateOf
40  import androidx.compose.runtime.remember
41  import androidx.compose.runtime.setValue
42  import androidx.compose.ui.Alignment
43  import androidx.compose.ui.Modifier
44  import androidx.compose.ui.res.painterResource
45  import androidx.compose.ui.res.stringResource
46  import androidx.compose.ui.text.input.ImeAction
47  import androidx.compose.ui.text.input.KeyboardType
```

```
48  import androidx.compose.ui.tooling.preview.Preview
49  import androidx.compose.ui.unit.dp
50  import com.example.tiptime.ui.theme.TipTimeTheme
51  import java.text.NumberFormat
52
53  class MainActivity : ComponentActivity() {
54      override fun onCreate(savedInstanceState: Bundle?) {
55          enableEdgeToEdge()
56          super.onCreate(savedInstanceState)
57          setContent {
58              TipTimeTheme {
59                  Surface(
60                      modifier = Modifier.fillMaxSize(),
61                  ) {
62                      TipTimeLayout()
63                  }
64              }
65          }
66      }
67  }
68
69  @Composable
70  fun TipTimeLayout() {
71      var amountInput by remember { mutableStateOf("") }
72      var tipInput by remember { mutableStateOf("") }
73      var roundUp by remember { mutableStateOf(false) }
74      val amount = amountInput.toDoubleOrNull() ?: 0.0
75      val tipPercent = tipInput.toDoubleOrNull() ?:0.0
76        val tip = calculateTip(amount = amount, tipPercent =
77  tipPercent, roundUp = roundUp)
78      val list = listOf("10", "15", "20")
79      Column(
80          modifier = Modifier
81              .statusBarsPadding()
82              .padding(horizontal = 40.dp)
83              .safeDrawingPadding()
84              .verticalScroll(rememberScrollState()),
85          horizontalAlignment = Alignment.CenterHorizontally,
86          verticalArrangement = Arrangement.Center
87      ) {
88          Text(
89              text = stringResource(R.string.calculate_tip),
90              modifier = Modifier
91                  .padding(bottom = 16.dp, top = 40.dp)
92                  .align(alignment = Alignment.Start)
93          )
94          EditNumberField(
95              value = amountInput,
96              onValueChange = { amountInput = it},
97              label = R.string.bill_amount,
98              keyboardOptions = KeyboardOptions.Default.copy(
99                  keyboardType = KeyboardType.Number,
```

```
                imeAction = ImeAction.Next
            ),
            leadingIcon = R.drawable.money,
            modifier = Modifier
                .padding(bottom = 32.dp)
                .fillMaxWidth()
        )
        EditDropDownField(
            label = R.string.tip_amount_label,
            leadingIcon = R.drawable.percent,
            list = list,
            preselected = tipInput,
            onSelected = { tipInput = it },
            modifier = Modifier
        )
        RoundTheTipRow(
            roundUp = roundUp,
            onRoundUpChanged = { roundUp = it },
            modifier = Modifier
                .padding(bottom = 32.dp)
        )
        Text(
            text = stringResource(R.string.tip_amount, tip),
            style = MaterialTheme.typography.displaySmall
        )
        Spacer(modifier = Modifier.height(150.dp))
    }
}

@Composable
fun EditNumberField(
    value: String,
    @StringRes label: Int,
    @DrawableRes leadingIcon: Int,
    onValueChange: (String) -> Unit,
    keyboardOptions: KeyboardOptions,
    modifier: Modifier = Modifier
) {
    TextField(
        value = value,
         leadingIcon = { Icon(painter = painterResource(id =
leadingIcon), null) },
        onValueChange = onValueChange,
        singleLine = true,
        label = {
            Text(text = stringResource(label))
        },
        keyboardOptions = keyboardOptions,
        modifier = modifier
    )
}

```

```
@Composable
fun EditDropDownField(
    list: List<String>,
    preselected: String,
    onSelected: (String) -> Unit,
    @StringRes label: Int,
    @DrawableRes leadingIcon: Int,
    modifier: Modifier = Modifier
) {
    var expanded by remember { mutableStateOf(false) }
                var    selectedList    by    remember    {
mutableStateOf(preselected) }
    Column {
        TextField(
            readOnly = true,
            value = selectedList,
            leadingIcon = {
                Icon(
                                painter = painterResource(id =
leadingIcon),
                    null
                )
            },
            onValueChange = { },
            singleLine = true,
            label = {
                Text(text = stringResource(label))
            },
            trailingIcon = {
                Icon(
                    Icons.Outlined.ArrowDropDown,
                    null,
                    modifier = Modifier.clickable { expanded
= !expanded })
            },
            modifier = modifier.fillMaxWidth()
        )
        DropdownMenu(
            expanded = expanded,
            onDismissRequest = {
                expanded = false
            }
        ) {
            list.forEach { item ->
                DropdownMenuItem(
                    onClick = {
                        selectedList = item

                        expanded = false
                        onSelected(item)
                    },
                    text = {
```

```
                            Text(text = item)
                        }
                    )
                }
            }
        }
    }
}

@Composable
fun RoundTheTipRow(
    roundUp: Boolean,
    onRoundUpChanged: (Boolean) -> Unit,
    modifier: Modifier = Modifier
) {
    Row(
        modifier = modifier
            .fillMaxWidth()
            .size(48.dp),
        verticalAlignment = Alignment.CenterVertically,
    ) {
        Text(text = stringResource(R.string.round_up_tip))
        Box(
            modifier = Modifier
                .fillMaxWidth()
        ) {
            Switch(
                checked = roundUp,
                onCheckedChange = onRoundUpChanged,
                modifier = Modifier
                    .align(Alignment.BottomEnd)
            )
        }
    }
}

private fun calculateTip(amount: Double, tipPercent: Double
= 15.0, roundUp: Boolean): String {
    var tip = tipPercent / 100 * amount
    if (roundUp) {
        tip = kotlin.math.ceil(tip)
    }
    return NumberFormat.getCurrencyInstance().format(tip)
}
@Preview(showBackground = true)
@Composable
fun TipTimeLayoutPreview() {
    TipTimeTheme {
        TipTimeLayout()
    }
}
```

Tabel 4. Source Code Jawaban Soal 1 Modul 2

XML
MainActivity.kt

```
1    package com.example.tiptimexml
2
3    import android.os.Bundle
4    import androidx.activity.ComponentActivity
5    import androidx.core.widget.doAfterTextChanged
6    import androidx.lifecycle.ViewModelProvider
7    import
8    com.example.tiptimexml.databinding.ActivityMainBinding
9
10   class MainActivity : ComponentActivity() {
11
12       private lateinit var viewModel: TipViewModel
13       private lateinit var binding: ActivityMainBinding
14
15       override fun onCreate(savedInstanceState: Bundle?) {
16           super.onCreate(savedInstanceState)
17                                                   binding      =
18   ActivityMainBinding.inflate(layoutInflater)
19           setContentView(binding.root)
20
21                                                 viewModel      =
22   ViewModelProvider(this)[TipViewModel::class.java]
23
24           val percentages = listOf(10, 15, 20)
25           binding.tipPercentageDropdown.setAdapter(
26                       TipViewModel.TipPercentageAdapter(this,
27   percentages)
28           )
29
30           fun updateTip() {
31                                                  val    bill    =
32   binding.billAmountEditText.text?.toString()
33                                               val   percent    =
34   binding.tipPercentageDropdown.text.toString().toIntOrNull()
35   ?: 0
36               val round = binding.roundUpSwitch.isChecked
37               viewModel.calculateTip(bill, percent, round)
38           }
39
40            binding.billAmountEditText.doAfterTextChanged    {
41   updateTip() }
42               binding.tipPercentageDropdown.doAfterTextChanged
43   { updateTip() }
44            binding.roundUpSwitch.setOnCheckedChangeListener {
45   _, _ -> updateTip() }
46
47                           viewModel.tipAmount.observe(this)   {
48   binding.tipResultText.text = it }
     }
```

```
    }
```

Tabel 5. Source Code Jawaban Soal 1 Modul 2

TipViewModel.kt

```
1   package com.example.tiptimexml
2
3   import android.content.Context
4   import android.widget.ArrayAdapter
5   import androidx.lifecycle.LiveData
6   import androidx.lifecycle.MutableLiveData
7   import androidx.lifecycle.ViewModel
8   import kotlin.math.ceil
9
10  class TipViewModel : ViewModel() {
11
12      private val _tipAmount = MutableLiveData<String>()
13      val tipAmount: LiveData<String> = _tipAmount
14
15          fun  calculateTip(bill:  String?,  percentage:  Int,
16  roundUp: Boolean) {
17          val billAmount = bill?.toDoubleOrNull() ?: 0.0
18          var tip = billAmount * percentage / 100.0
19          if (roundUp) tip = ceil(tip)
20          _tipAmount.value = "Tip Amount: $%.2f".format(tip)
21      }
22
23         class  TipPercentageAdapter(context:  Context,  items:
24  List<Int>) :
25                                      ArrayAdapter<Int>(context,
26  android.R.layout.simple_dropdown_item_1line, items)
27  }
```

Tabel 6. Source Code Jawaban Soal 1 Modul 2

activity_main.xml

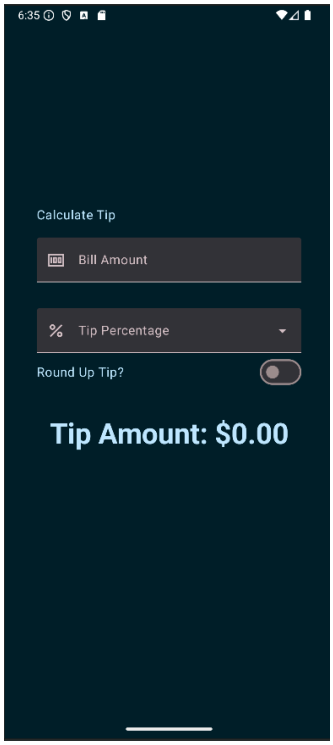
```
1   <?xml version="1.0" encoding="utf-8"?>
2   <androidx.constraintlayout.widget.ConstraintLayout
3   xmlns:android="http://schemas.android.com/apk/res/android"
4       xmlns:app="http://schemas.android.com/apk/res-auto"
5       xmlns:tools="http://schemas.android.com/tools"
6       android:layout_width="match_parent"
7       android:layout_height="match_parent"
8       android:padding="24dp"
9       android:background="#002B36">
10
11      <TextView
12          android:id="@+id/title"
13          android:layout_width="wrap_content"
```

```
14          android:layout_height="wrap_content"
15          android:layout_marginTop="60dp"
16          android:text="@string/calculate_tip"
17          android:textColor="#FFFFFF"
18          android:textSize="16sp"
19          app:layout_constraintStart_toStartOf="parent"
20          app:layout_constraintTop_toTopOf="parent" />
21
22      <com.google.android.material.textfield.TextInputLayout
23          android:id="@+id/billInputLayout"
24
25  style="@style/Widget.MaterialComponents.TextInputLayout.Outl
26  inedBox.Dense"
27          android:layout_width="0dp"
28          android:layout_height="wrap_content"
29          app:layout_constraintTop_toBottomOf="@id/title"
30          app:layout_constraintStart_toStartOf="parent"
31          app:layout_constraintEnd_toEndOf="parent"
32          android:layout_marginTop="16dp"
33          app:startIconDrawable="@drawable/money"
34          app:startIconTint="@color/light_pink">
35
36
37  <com.google.android.material.textfield.TextInputEditText
38              android:id="@+id/billAmountEditText"
39              android:layout_width="match_parent"
40              android:layout_height="wrap_content"
41              android:hint="@string/bill_amount"
42              android:inputType="numberDecimal"
43              android:paddingStart="40dp"
44              tools:ignore="RtlSymmetry" />
45      </com.google.android.material.textfield.TextInputLayout>
46
47      <com.google.android.material.textfield.TextInputLayout
48          android:id="@+id/tipDropdownLayout"
49
50  style="@style/Widget.MaterialComponents.TextInputLayout.Outl
51  inedBox.Dense.ExposedDropdownMenu"
52          android:layout_width="0dp"
53          android:layout_height="wrap_content"
54          android:hint="@string/tip_percentage"
55
56  app:layout_constraintTop_toBottomOf="@id/billInputLayout"
57          app:layout_constraintStart_toStartOf="parent"
58          app:layout_constraintEnd_toEndOf="parent"
59          android:layout_marginTop="16dp"
60          app:startIconDrawable="@drawable/percent"
61          app:startIconTint="@color/light_pink"
62          >
63
64          <AutoCompleteTextView
65              android:id="@+id/tipPercentageDropdown"
```

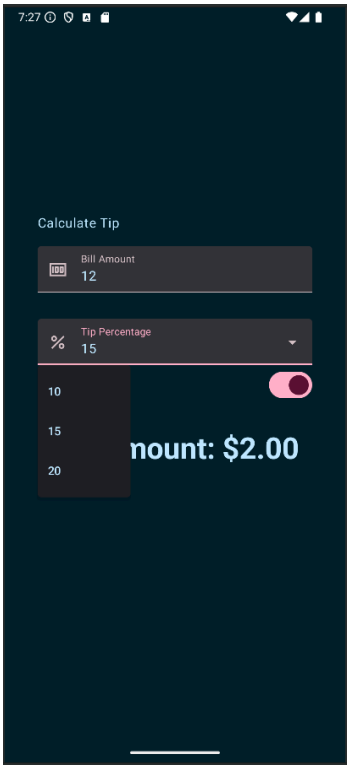
```
66  style="@style/Widget.MaterialComponents.TextInputLayout.Outl
67  inedBox"
68              android:layout_width="match_parent"
69              android:layout_height="wrap_content"
70              android:inputType="none"
71              android:paddingStart="40dp"
72  tools:ignore="LabelFor,RtlSymmetry,SpeakableTextPresentCheck
73  " />
74      </com.google.android.material.textfield.TextInputLayout>
75
76      <TextView
77          android:id="@+id/roundUpLabel"
78          android:layout_width="wrap_content"
79          android:layout_height="wrap_content"
80          android:text="@string/round_up_tip"
81          android:textColor="#FFFFFF"
82          android:layout_marginTop="12dp"
83
84  app:layout_constraintTop_toBottomOf="@id/tipDropdownLayout"
85          app:layout_constraintStart_toStartOf="parent"/>
86
87
88  <com.google.android.material.switchmaterial.SwitchMaterial
89          android:id="@+id/roundUpSwitch"
90          android:layout_width="wrap_content"
91          android:layout_height="wrap_content"
92          app:layout_constraintTop_toTopOf="@id/roundUpLabel"
93          app:layout_constraintEnd_toEndOf="parent"
94          app:thumbTint="@color/switch_thumb_color"
95          app:trackTint="@color/switch_track_color"
96          app:useMaterialThemeColors="false"/>
97      <TextView
98          android:id="@+id/tipResultText"
99          android:layout_width="0dp"
100         android:layout_height="wrap_content"
101         android:text="@string/tip_amount_result"
102         android:textSize="24sp"
103         android:textStyle="bold"
104         android:textColor="#B3E5FC"
105         android:gravity="center"
106
107 app:layout_constraintTop_toBottomOf="@id/roundUpSwitch"
108         app:layout_constraintStart_toStartOf="parent"
109         app:layout_constraintEnd_toEndOf="parent"
110         android:layout_marginTop="32dp"/>
111 </androidx.constraintlayout.widget.ConstraintLayout>
```

Tabel 7. Source Code Jawaban Soal 1 Modul 2

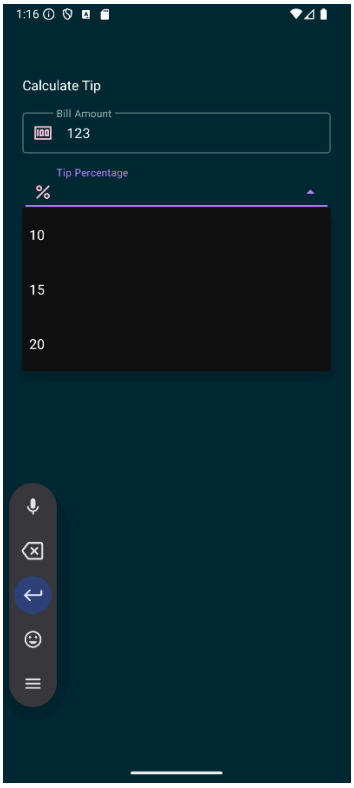
B. Output Program



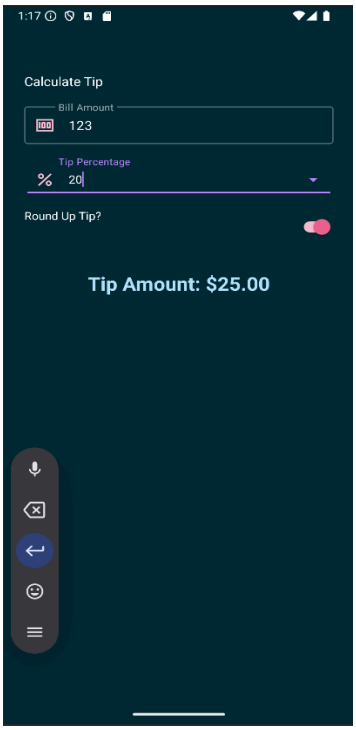
Gambar 7. Screenshot Hasil Jawaban Soal 1 Compose Modul 2



Gambar 8. Screenshot Hasil Jawaban Soal 1 Compose Modul 2



Gambar 9. Screenshot Hasil Jawaban Soal 1 XML Modul 2



Gambar 10. Screenshot Hasil Jawaban Soal 1 Compose Modul 2

C. Pembahasan

Compose
MainActivity.kt:

Pada line 93 - 105, fungsi EditNumberField dipanggil dengan atribut label untuk memberikan label, dan keyboardoptions untuk mengatur enter menjadi next, dan keyboard jenis numeral, dan atribut leadingicon untuk menampilkan drawable
Pada line 106 - 113, fungsi EditDropDownField dipanggil dengan atribut list untuk memasukan list apa percentage, preselect untuk menjadi bridge untuk menampilkan value, onselect untuk melakukan kalkulasi tip
Pada line 161 - 163, dideklarasikan variable expanded dan selectedList by remember dengan mutablestateof untuk menyimpan state data dinamis yang diperlukan
Pada line 165 - 188, Class textfiled dipanggil untuk menampung input dari user, dan atribut trailingicon yang dapat menerima respons lambda berisi class Icon yang berupa drawable dan modifier clickable untuk mengubah state expanded
Pada line 189 - 194, class dropdownmenu dipanggil dengan atribut expanded memiliki value false
Pada line 195 - 207, terdapat perulangan untuk memanggil class dropdownitem sebanyak item yang ada pada list

XML
MainActivity.kt:

Pada line 16, diimport binding untuk bisa membuat layout
Pada line 19 - 20, diimport viewmodel untuk melakukan pengolahan data dan adapter untuk activity_main
Pada line 28 - 36, fungsi updatetip dibuat untuk melakukan update state dari tip dan memanggil fungsi kalkulasi dari viewmodel
Pada line 38 - 43, terdapat binding eventlistener yang akan menjalankan fungsi updatetip jika kondisi terpenuhi

TipViewModel.kt:

Pada line 10 - 21, class TipViewModel dideklarasi untuk melakukan kalkulasi dan mengolah data yang diperlukan oleh view
Pada line 23 - 27, class TipPercentageAdapter dideklarasi dengan parameter context untuk menerima binding dropdown layout dari activity_main dan list untuk menerima list dan item-itemnya, class ini berfungsi untuk melakukan generate layout dropdownitem sebanyak item yang ada pada list

activity_main.xml:

Pada line 11 - 20, dideklarasikan komponen textview untuk title dari aplikasi dengan margin top 60dp
Pada line 22 - 45, dideklarasikan layout TextInputLayout yang berisi TextInputEditText component yang digunakan untuk menampilkan form input untuk jumlah bill

Pada line 47 - 73, dideklarasikan layout TextInputLayout yang memiliki style TextInputLayout.OutlinedBox.ExposedDropdownMenu dan berisi komponen AutoCompleteTextView
Pada line 88 - 96, dideklarasikan komponen switchmaterial untuk menampilkan bentuk switch yang berguna sebagai input boolean

D. Tautan Git

Berikut adalah tautan untuk source code yang telah dibuat.
https://github.com/biahlil/Pemrograman-Mobile.git

MODUL 3

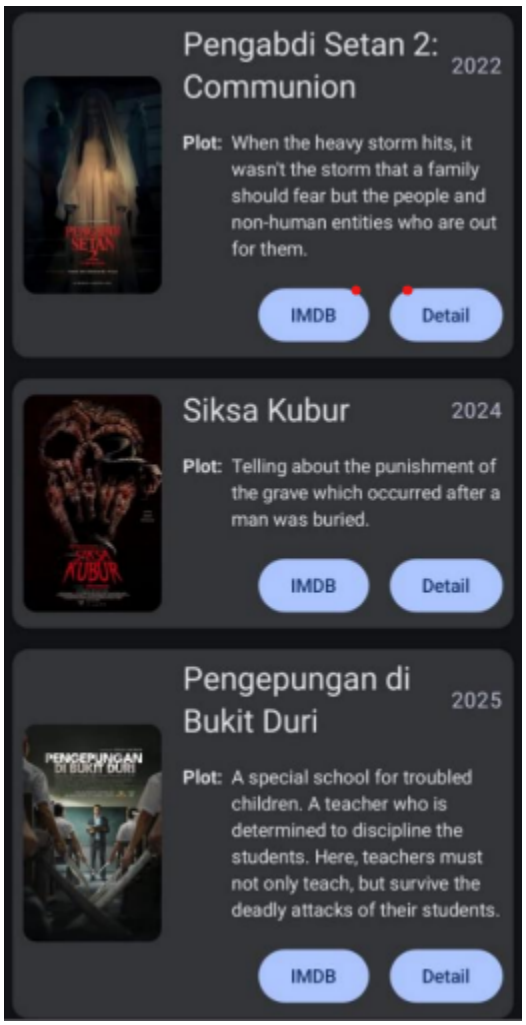
SOAL

Soal Praktikum:

1. Buatlah sebuah aplikasi Android menggunakan XML dan Jetpack Compose yang dapat menampilkan list dengan ketentuan berikut:
    a. List menggunakan fungsi RecyclerView (XML) dan LazyColumn (Compose)
    b. List paling sedikit menampilkan 5 item. Tema item yang ingin ditampilkan bebas
    c. Item pada list menampilkan teks dan gambar sesuai dengan contoh di bawah
    d. Terdapat 2 button dalam list, dengan fungsi berikut:
        i. Button pertama menggunakan intent eksplisit untuk membuka aplikasi atau browser lain
        ii. Button kedua menggunakan Navigation component untuk membuka laman detail item
    e. Sudut item pada list dan gambar di dalam list melengkung atau rounded corner menggunakan Radius
    f. Saat orientasi perangkat berubah/dirotasi, baik ke portrait maupun landscape, aplikasi responsif dan dapat menunjukkan list dengan baik. Data di dalam list tidak boleh hilang
    g. Aplikasi menggunakan arsitektur single activity (satu activity memiliki beberapa fragment)
    h. Aplikasi berbasis XML harus menggunakan ViewBinding

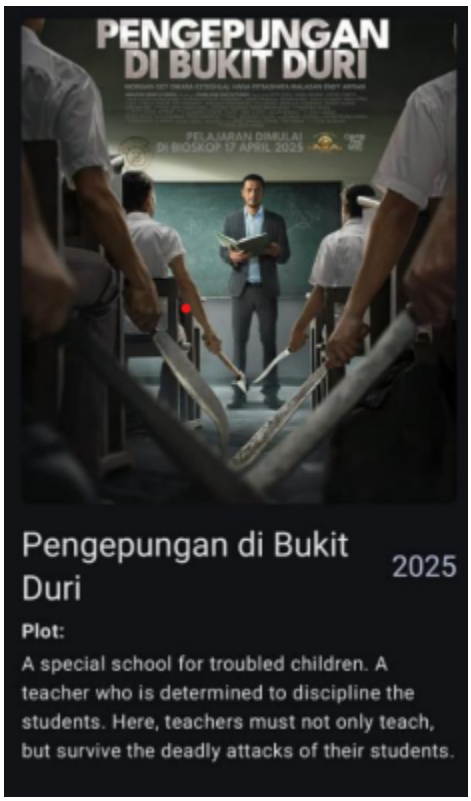
2. Mengapa RecyclerView masih digunakan, padahal RecyclerView memiliki kode yang panjang dan bersifat boiler-plate, dibandingkan LazyColumn dengan kode yang lebih singkat?

    UI item list harus berisi 1 gambar, 2 button (intent eksplisit dan navigasi), dan 2 baris teks dan setiap baris memiliki 2 teks yang berbeda. Diusahakan agar desain UI item list menyerupai UI berikut:



Gambar 11. Contoh UI List

Desain UI laman detail bebas, tetapi diusahakan untuk mengikuti kaidah desain Material Design dan data item ditampilkan penuh di laman detail seperti contoh berikut:



Gambar 12. Contoh UI Detail

A. Source Code

Compose
MovieListViewModel.kt

```
1   package com.example.movielist.ui.movielist
2
3   import androidx.compose.runtime.State
4   import androidx.compose.runtime.mutableStateOf
5   import androidx.lifecycle.ViewModel
6   import androidx.lifecycle.viewModelScope
7   import com.example.movielist.data.Result
8   import com.example.movielist.data.repository.MovieRepository
9   import
10  com.example.movielist.data.repository.impl.FakeMovieReposito
11  ry
12  import com.example.movielist.domain.model.Movie
13  import kotlinx.coroutines.flow.MutableSharedFlow
14  import kotlinx.coroutines.flow.asSharedFlow
15  import kotlinx.coroutines.launch
16  import timber.log.Timber
17
18
19  class MovieListViewModel(
20              val   movieRepository   :   MovieRepository   =
21  FakeMovieRepository()
22  ) : ViewModel() {
23
24      private val _eventFlow = MutableSharedFlow<UiEvent>()
25      val eventFlow = _eventFlow.asSharedFlow()
26
27                      private     val     _movieListState     =
28  mutableStateOf(MovieListState())
29              val   movieListState:   State<MovieListState>   =
30  _movieListState
31
32      init {
33          getAllMovies()
34      }
35
36      fun getAllMovies() {
37          viewModelScope.launch {
38              _movieListState.value = MovieListState(isLoading
39  = true)
40
41              movieRepository.getAllMovies().collect { result
42  ->
43                  when (result) {
44                      is Result.Success<List<Movie>> -> {
45                                          _movieListState.value =
46  MovieListState(movies = result.data, isLoading = false)
47
```

```
48
49  Timber.tag("MovieListViewModel").i("Movies:
50  ${_movieListState.value}")
51
52                      }
53                      is Result.Error -> {
54                                          _movieListState.value =
55  MovieListState(isLoading = false)
56                          _eventFlow.emit(
57                              UiEvent.ShowSnackbar(
58                                                      message =
59  result.exception.message ?: "Unknown Error"
60                              )
61                          )
62                      }
63                  }
64              }
65          }
66      }
67
68      fun onOpenUrlClicked(url: String) {
69          viewModelScope.launch {
70              if (url.isNotBlank()) {
71                  _eventFlow.emit(UiEvent.OpenUrl(url))
72                    Timber.tag("MovieListViewModel").i("intent
73  into URL: $url")
74              } else {
75                  _eventFlow.emit(UiEvent.ShowSnackbar("URL is
76  not valid"))
77              }
78          }
79      }
80  }
81
82  data class MovieListState(
83      val movies: List<Movie> = emptyList(),
84      val isLoading: Boolean = true
85  )
86
87  sealed class UiEvent {
88      data class ShowSnackbar(val message: String) : UiEvent()
89      data class OpenUrl(val url: String) : UiEvent()
90  }
```

Tabel 8. Source Code Jawaban Soal 1 Modul 3

MovieListScreen.kt

```
1   package com.example.movielist.ui.movielist
2
3   import android.annotation.SuppressLint
4   import android.content.ActivityNotFoundException
```

```
5   import android.content.Intent
6   import androidx.compose.foundation.layout.Arrangement
7   import androidx.compose.foundation.layout.Box
8   import androidx.compose.foundation.layout.PaddingValues
9   import androidx.compose.foundation.layout.fillMaxSize
10  import androidx.compose.foundation.layout.padding
11  import androidx.compose.foundation.lazy.LazyColumn
12  import androidx.compose.foundation.lazy.items
13  import androidx.compose.material3.CircularProgressIndicator
14  import androidx.compose.material3.Scaffold
15  import androidx.compose.material3.SnackbarDuration
16  import androidx.compose.material3.SnackbarHost
17  import androidx.compose.material3.SnackbarHostState
18  import androidx.compose.material3.Surface
19  import androidx.compose.runtime.Composable
20  import androidx.compose.runtime.LaunchedEffect
21  import androidx.compose.runtime.remember
22  import androidx.compose.ui.Alignment
23  import androidx.compose.ui.Modifier
24  import androidx.compose.ui.platform.LocalContext
25  import androidx.compose.ui.res.stringResource
26  import androidx.compose.ui.tooling.preview.Preview
27  import androidx.compose.ui.unit.dp
28  import androidx.core.net.toUri
29  import com.example.movielist.domain.model.Movie
30  import com.example.movielist.ui.components.MovieCard
31  import com.example.movielist.ui.theme.MovieListTheme
32  import kotlinx.coroutines.flow.collectLatest
33  import timber.log.Timber
34
35  @Composable
36  fun MovieListScreen(
37      viewModel: MovieListViewModel,
38      detailOnclick: (Movie) -> Unit,
39      modifier: Modifier = Modifier
40  ) {
41      val state = viewModel.movieListState.value
42      val context = LocalContext.current
43      val snackbarHostState = remember { SnackbarHostState() }
44
45      LaunchedEffect(key1 = true) {
46          viewModel.eventFlow.collectLatest { event ->
47              when (event) {
48                  is UiEvent.ShowSnackbar -> {
49                      snackbarHostState.showSnackbar(
50                          message = event.message,
51                          duration = SnackbarDuration.Short
52                      )
53                  }
54                  is UiEvent.OpenUrl -> {
55                      try {
56
```

```
                                                val intent =
Intent(Intent.ACTION_VIEW, event.url.toUri())
                            context.startActivity(intent)
                        } catch (e: ActivityNotFoundException) {
                            Timber.tag("MovieListScreen")
                                                .e("Error opening URL:
${e.message} ${event.url}")
                        }
                    }
                }
            }
        }
        Scaffold(
            snackbarHost = { SnackbarHost(snackbarHostState) },
            modifier = modifier
        ) { paddingValues ->
            Box(
                modifier = Modifier
                    .fillMaxSize()
                    .padding(paddingValues)
            ) {
                // Tampilkan loading
                if (state.isLoading) {
                            CircularProgressIndicator(modifier =
Modifier.align(Alignment.Center))
                } else {
                    LazyColumn(
                        contentPadding = PaddingValues(16.dp),
                                            verticalArrangement =
Arrangement.spacedBy(16.dp)
                    ) {
                        items(state.movies) { movie ->
                                                val imdbUrl =
stringResource(movie.imdbLink)
                            MovieCard(
                                image = movie.image,
                                titleText = movie.titleText,
                                yearText = movie.yearText,
                                                descriptionText =
movie.descriptionText,
                                openBrowserClick = {

viewModel.onOpenUrlClicked(imdbUrl)
                                },
                                detailOnclick = {
                                    detailOnclick(movie)
                                }
                            )
                            Timber.tag("MovieProperty").d("Movie
Detail: $movie")
                        }
                    }
```

```
109             }
110         }
111     }
112 }
113
114 @SuppressLint("ViewModelConstructorInComposable")
115 @Preview
116 @Composable
117 private fun MovieListScreenPrev() {
118     MovieListTheme(darkTheme = true) {
119         Surface {
120                                     MovieListScreen(viewModel   =
121 MovieListViewModel(), detailOnclick = {})
123         }
124     }
125 }
```

Tabel 9. Source Code Jawaban Soal 1 Modul 3

MovieDetailViewModel.kt

```
1   package com.example.movielist.ui.moviedetail
2
3   import androidx.compose.runtime.State
4   import androidx.compose.runtime.mutableStateOf
5   import androidx.lifecycle.ViewModel
6   import androidx.lifecycle.viewModelScope
7   import com.example.movielist.data.Result
8   import com.example.movielist.data.repository.MovieRepository
9   import
10  com.example.movielist.data.repository.impl.FakeMovieReposito
11  ry
12  import com.example.movielist.domain.model.Movie
13  import com.example.movielist.ui.movielist.UiEvent
14  import kotlinx.coroutines.flow.MutableSharedFlow
15  import kotlinx.coroutines.flow.asSharedFlow
16  import kotlinx.coroutines.launch
17  import timber.log.Timber
18
19
20  class MovieDetailViewModel(
21          private  val  movieRepository:  MovieRepository  =
22  FakeMovieRepository(),
23  ) : ViewModel() {
24                  private    val    _movieDetailState    =
25  mutableStateOf(MovieDetailState())
26          val  movieDetailState:  State<MovieDetailState>  =
27  _movieDetailState
28
29                      private     val     _eventFlow     =
30  MutableSharedFlow<UiEvent.ShowSnackbar>()
31      val eventFlow = _eventFlow.asSharedFlow()
32
```

```
    fun getMovieDetail(id: Int?) {
        if (id == null) {
            viewModelScope.launch {
                _eventFlow.emit(UiEvent.ShowSnackbar("Movie ID tidak valid."))
            }
            return
        }
        viewModelScope.launch {
                                    _movieDetailState.value = MovieDetailState(isLoading = true)

            movieRepository.getMovie(id).collect { result ->
                when (result) {
                    is Result.Success -> {
                                        _movieDetailState.value = MovieDetailState(
                            movie = result.data,
                            isLoading = false
                        )

                    Timber.tag("MovieDetailViewModel").i("Get into Movie Detail: ${_movieDetailState.value}")
                    }

                    is Result.Error -> {
                                        _movieDetailState.value = MovieDetailState(isLoading = false)
                        _eventFlow.emit(
                            UiEvent.ShowSnackbar(
                                                message = result.exception.message ?: "Movie not found"
                            )
                        )
                    }
                }
            }
        }
    }
}

data class MovieDetailState(
    val movie: Movie? = null,
    val isLoading: Boolean = true
)
```

Tabel 10. Source Code Jawaban Soal 1 Modul 3

FakeRepository.kt

```
package com.example.movielist.data.repository.impl

import com.example.movielist.data.Result
```

```
4   import com.example.movielist.data.repository.MovieRepository
5   import com.example.movielist.domain.model.Movie
6   import kotlinx.coroutines.delay
7   import kotlinx.coroutines.flow.Flow
8   import kotlinx.coroutines.flow.flow
9   import kotlinx.io.IOException
10
11  class FakeMovieRepository : MovieRepository {
12
13        private val movies = listOf(venom, kiki, dayLater,
14  turningRed, vhs, castle)
15
16      override fun getMovie(postId: Int?): Flow<Result<Movie>>
17  {
18          return flow {
19              delay(1000)
20
21              if (postId == null) {
22                   emit(Result.Error(Exception("Movie ID can't
23  be null")))
24                  return@flow // Hentikan eksekusi flow
25              }
26
27               val movieById: Movie? = movies.find { it.id ==
28  postId }
29
30              if (movieById != null) {
31                  emit(Result.Success(movieById))
32              } else {
33                    emit(Result.Error(Exception("Movie with id
34  $postId not found")))
35              }
36          }
37      }
38
39                  override    suspend    fun    getAllMovies():
40  Flow<Result<List<Movie>>> {
41          return flow {
42              try {
43                  delay(1000)
44                  emit(Result.Success(movies))
45              } catch (e: IOException) {
46                  emit(Result.Error(Exception(e.message)))
47              }
48          }
49      }
50  }
```

Tabel 11. Source Code Jawaban Soal 1 Modul 3

XML
MainActivity.kt

```
1   package com.example.movielistxml
2
3   import androidx.appcompat.app.AppCompatActivity
4   import android.os.Bundle
5   import androidx.navigation.NavController
6   import androidx.navigation.findNavController
7   import androidx.navigation.fragment.NavHostFragment
8   import
9   androidx.navigation.ui.setupActionBarWithNavController
10  import
11  com.example.movielistxml.databinding.ActivityMainBinding
12
13  class MainActivity : AppCompatActivity() {
14
15      private lateinit var binding: ActivityMainBinding
16      private lateinit var navController: NavController
17
18      override fun onCreate(savedInstanceState: Bundle?) {
19          super.onCreate(savedInstanceState)
20                                                  binding      =
21  ActivityMainBinding.inflate(layoutInflater)
22          setContentView(binding.root)
23
24          val navHostFragment = supportFragmentManager
25                  .findFragmentById(R.id.nav_host_fragment) as
26  NavHostFragment // Use the ID from your XML
27          navController = navHostFragment.navController
28
29          setupActionBarWithNavController(navController)
30      }
31
32      override fun onSupportNavigateUp(): Boolean {
33                                      val    navController    =
34  findNavController(R.id.nav_host_fragment)
35                      return   navController.navigateUp()   ||
36  super.onSupportNavigateUp()
        }
    }
```

Tabel 12. Source Code Jawaban Soal 1 Modul 3

MovieListAdapter.kt

```
1   package com.example.movielistxml
2
3   import android.view.LayoutInflater
4   import android.view.ViewGroup
5   import androidx.core.content.ContextCompat.getString
6   import androidx.recyclerview.widget.DiffUtil
7   import androidx.recyclerview.widget.ListAdapter
8   import androidx.recyclerview.widget.RecyclerView
9   import com.example.movielistxml.databinding.ItemMovieBinding
10  import com.example.movielistxml.domain.MovieModel
```

```
11
12  class MovieListAdapter(
13      private val onDetailClick: (MovieModel) -> Unit,
14      private val onBrowserClick: (MovieModel) -> Unit
15  )                   :                   ListAdapter<MovieModel,
16  MovieListAdapter.MovieViewHolder>(MovieDiffCallback()) {
17
18          override  fun  onCreateViewHolder(parent:  ViewGroup,
19  viewType: Int): MovieViewHolder {
20                                              val     binding     =
21  ItemMovieBinding.inflate(LayoutInflater.from(parent.context)
22  , parent, false)
23          return MovieViewHolder(binding)
24      }
25
26       override fun onBindViewHolder(holder: MovieViewHolder,
27  position: Int) {
28          val movie = getItem(position)
29          holder.bind(movie)
30      }
31
32          inner  class  MovieViewHolder(private  val  binding:
33  ItemMovieBinding) :
34          RecyclerView.ViewHolder(binding.root) {
35          fun bind(movie: MovieModel) {
36              val context = binding.root.context
37                          val  titleText  =  getString(context,
38  movie.titleText)
39                           val  yearText  =  getString(context,
40  movie.yearText)
41                      val descriptionText = getString(context,
42  movie.descriptionText)
43
44
45  binding.ivMoviePoster.setImageResource(movie.image)
46              binding.tvMovieTitle.text = titleText
47              binding.tvMovieYear.text = yearText
48                              binding.tvMovieDescription.text  =
49  descriptionText
50              binding.btnItemDetail.setOnClickListener {
51                  onDetailClick(movie)
52              }
53
54              binding.btnItemBrowser.setOnClickListener {
55                  onBrowserClick(movie)
56              }
57          }
58      }
59  }
60
61  class                    MovieDiffCallback                    :
62  DiffUtil.ItemCallback<MovieModel>() {
```

```
63          override fun areItemsTheSame(oldItem: MovieModel,
64  newItem: MovieModel): Boolean {
65          return oldItem.id == newItem.id
66      }
67
68        override fun areContentsTheSame(oldItem: MovieModel,
69  newItem: MovieModel): Boolean {
70          return oldItem == newItem
        }
    }
```

Tabel 13. Source Code Jawaban Soal 1 Modul 3

MovieListFragment.kt

```
1   package com.example.movielistxml.ui.movielist
2
3   import android.content.Intent
4   import android.os.Bundle
5   import android.view.LayoutInflater
6   import android.view.View
7   import android.view.ViewGroup
8   import android.widget.Toast
9   import androidx.core.net.toUri
10  import androidx.core.view.isVisible
11  import androidx.fragment.app.Fragment
12  import androidx.fragment.app.viewModels
13  import androidx.lifecycle.ViewModel
14  import androidx.lifecycle.ViewModelProvider
15  import androidx.navigation.fragment.findNavController
16  import com.example.movielistxml.MovieListAdapter
17  import
18  com.example.movielistxml.data.repository.MovieRepository
19  import
20  com.example.movielistxml.data.repository.impl.FakeMovieRepos
21  itory
22  import
23  com.example.movielistxml.databinding.FragmentMovieListBindin
24  g
25  import timber.log.Timber
26
27  class MovieListFragment : Fragment() {
28
29      private var _binding: FragmentMovieListBinding? = null
30      private val binding get() = _binding!!
31
32       private val viewModel: MovieListViewModel by viewModels
33  {
34          MovieListViewModelFactory(FakeMovieRepository())
35      }
36
37      private lateinit var movieAdapter: MovieListAdapter
38
```

```
39      override fun onCreateView(
40          inflater: LayoutInflater, container: ViewGroup?,
41          savedInstanceState: Bundle?
42      ): View {
43                                                  _binding     =
44  FragmentMovieListBinding.inflate(inflater, container, false)
45              Timber.tag("MovieListFragment").d("onCreateView:
46  Binding inflated.")
47          return binding.root
48      }
49
50                override    fun    onViewCreated(view:    View,
51  savedInstanceState: Bundle?) {
52          super.onViewCreated(view, savedInstanceState)
53             Timber.tag("MovieListFragment").d("onViewCreated:
54  View is created.")
55
56          setupRecyclerView()
57          observeViewModel()
58      }
59
60      private fun setupRecyclerView() {
61          movieAdapter = MovieListAdapter(
62              onDetailClick = { movie ->
63                  // Logika untuk tombol Detail
64                          Timber.d("Detail  button  clicked  for:
65  ${movie.titleText}")
66                                                  val   action   =
67  MovieListFragmentDirections.actionMovieListFragmentToMovieDe
68  tailFragment(movie.id)
69                  findNavController().navigate(action)
70              },
71              onBrowserClick = { movie ->
72                          Timber.d("Browser  button  clicked  for:
73  ${movie.titleText}")
74                  val imdbUrl = getString(movie.imdbLink)
75                       val  intent  =  Intent(Intent.ACTION_VIEW,
76  imdbUrl.toUri())
77                  try {
78                      startActivity(intent)
79                  } catch (e: Exception) {
80                                  Toast.makeText(requireContext(),
81  e.message, Toast.LENGTH_SHORT).show()
82                  }
83              }
84          )
85          binding.rvMovies.adapter = movieAdapter
86
87  Timber.tag("MovieListFragment").d("setupRecyclerView:
88  RecyclerView and Adapter are set up.")
89      }
90
```

```
91      private fun observeViewModel() {
92                viewModel.movies.observe(viewLifecycleOwner)  {
93  movies ->
94              if (movies.isNotEmpty()) {
95                  Timber.tag("MovieListFragment").i("Observing
96  movies: ${movies.size} items submitted to adapter.")
97                  movieAdapter.submitList(movies)
98                                      binding.rvMovies.isVisible  =
99  movies.isNotEmpty()
100             }
101         }
102            viewModel.isLoading.observe(viewLifecycleOwner)  {
103 isLoading ->
104                 Timber.tag("MovieListFragment").d("Observing
105 isLoading: $isLoading")
106             binding.listProgressBar.isVisible = isLoading
107             if (isLoading) {
108                 binding.rvMovies.isVisible = false
109                    Timber.tag("MovieListFragment").d("Making
110 rvMovies invisible")
111
112             }
113         }
114         Timber.tag("MovieListFragment").d("observeViewModel:
115 Observers are set.")
116     }
117
118     override fun onDestroyView() {
119         super.onDestroyView()
120         _binding = null
121            Timber.tag("MovieListFragment").d("onDestroyView:
123 Binding is now null.")
124     }
125 }
126
127 class   MovieListViewModelFactory(private   val   repository:
128 MovieRepository) : ViewModelProvider.Factory {
129         override  fun  <T  :  ViewModel>  create(modelClass:
130 Class<T>): T {
131                                                             if
132 (modelClass.isAssignableFrom(MovieListViewModel::class.java)
133 ) {
134             @Suppress("UNCHECKED_CAST")
135             return MovieListViewModel(repository) as T
136         }
137            throw IllegalArgumentException("Unknown ViewModel
    class")
    }
}
```

Tabel 14. Source Code Jawaban Soal 1 Modul 3

MovieDetailFragment.kt

```
1   package com.example.movielistxml.ui.moviedetail
2
3   import androidx.lifecycle.ViewModel
4   import androidx.lifecycle.ViewModelProvider
5   import
6   com.example.movielistxml.data.repository.MovieRepository
7   import android.os.Bundle
8   import android.view.LayoutInflater
9   import android.view.View
10  import android.view.ViewGroup
11  import androidx.core.view.isVisible
12  import androidx.fragment.app.Fragment
13  import androidx.fragment.app.viewModels
14  import androidx.navigation.fragment.navArgs
15  import
16  com.example.movielistxml.data.repository.impl.FakeMovieRepos
17  itory
18  import
19  com.example.movielistxml.databinding.FragmentMovieDetailBind
20  ing
21  import timber.log.Timber
22  import kotlin.getValue
23
24  class MovieDetailFragment : Fragment() {
25
26      private var _binding: FragmentMovieDetailBinding? = null
27      private val binding get() = _binding!!
28
29            private  val  viewModel:  MovieDetailViewModel  by
30  viewModels {
31          MovieDetailViewModelFactory(FakeMovieRepository())
32      }
33
34      private val args: MovieDetailFragmentArgs by navArgs()
35
36      override fun onCreateView(
37          inflater: LayoutInflater, container: ViewGroup?,
38          savedInstanceState: Bundle?
39      ): View {
40                                                  _binding     =
41  FragmentMovieDetailBinding.inflate(inflater,      container,
42  false)
43            Timber.tag("MovieDetailFragment").d("onCreateView:
44  Binding inflated.")
45          return binding.root
46      }
47
48                override   fun   onViewCreated(view:   View,
49  savedInstanceState: Bundle?) {
50          super.onViewCreated(view, savedInstanceState)
51           Timber.tag("MovieDetailFragment").d("onViewCreated:
52  View is created.")
```

```
53          viewModel.getMovieDetail(args.movieId)
54          observeViewModel()
55      }
56
57      private fun observeViewModel() {
58           viewModel.movie.observe(viewLifecycleOwner) { movie
59  ->
60              movie?.let {
61
62  Timber.tag("MovieDetailFragment").i("Observing       movies:
63  $movie items submitted to adapter.")
64
65  binding.ivDetailPoster.setImageResource(it.image)
66                                      binding.tvDetailTitle.text  =
67  getString(it.titleText)
68                                       binding.tvDetailYear.text  =
69  getString(it.yearText)
70                                 binding.tvDetailDescription.text  =
71  getString(it.descriptionText)
72              }
73          }
74             viewModel.isLoading.observe(viewLifecycleOwner)  {
75  isLoading ->
76                     Timber.tag("MovieListFragment").d("Observing
77  isLoading: $isLoading")
78              binding.detailProgressBar.isVisible = isLoading
79
80              binding.contentScrollView.isVisible = !isLoading
81                     Timber.tag("MovieListFragment").d("Making
82  rvMovies invisible")
83          }
84      }
85
86      override fun onDestroyView() {
87          super.onDestroyView()
88          _binding = null
89      }
90  }
91
92  class  MovieDetailViewModelFactory(private  val  repository:
93  MovieRepository) : ViewModelProvider.Factory {
94          override  fun  <T  :  ViewModel>  create(modelClass:
95  Class<T>): T {
96                                                                  if
97  (modelClass.isAssignableFrom(MovieDetailViewModel::class.jav
98  a)) {
99              @Suppress("UNCHECKED_CAST")
100             return MovieDetailViewModel(repository) as T
101         }
102            throw  IllegalArgumentException("Unknown  ViewModel
103 class")
104     }
```

```
105 }
106
```

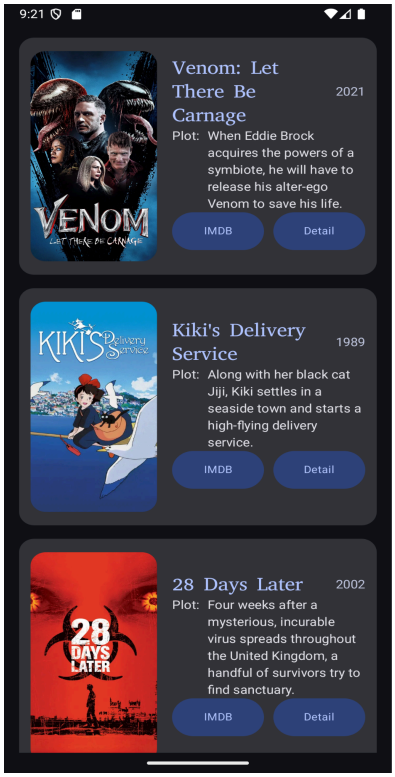
Tabel 15. Source Code Jawaban Soal 1 Modul 3

nav_graph.xml

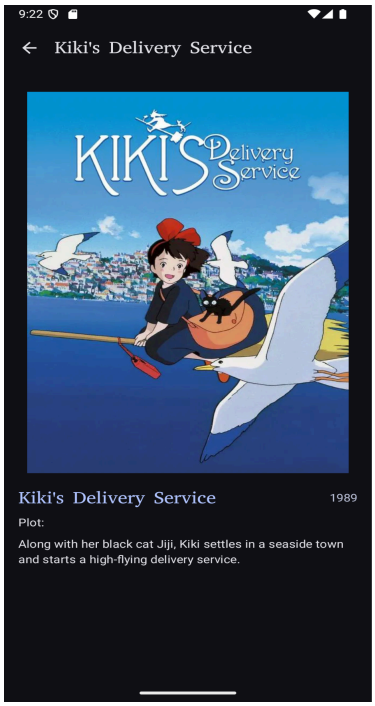
```
1   <?xml version="1.0" encoding="utf-8"?>
2
3   <navigation
4   xmlns:android="http://schemas.android.com/apk/res/android"
5   xmlns:app="http://schemas.android.com/apk/res-auto"
6   xmlns:tools="http://schemas.android.com/tools"
7   android:id="@+id/nav_graph"
8   app:startDestination="@id/movieListFragment">
9
10      <fragment
11          android:id="@+id/movieListFragment"
12
13  android:name="com.example.movielistxml.ui.movielist.MovieLis
14  tFragment">
15          <action
16
17  android:id="@+id/action_movieListFragment_to_movieDetailFrag
18  ment"
19              app:destination="@id/movieDetailFragment" />
20      </fragment>
21
22      <fragment
23          android:id="@+id/movieDetailFragment"
24
25  android:name="com.example.movielistxml.ui.moviedetail.MovieD
26  etailFragment"
27          tools:layout="@layout/fragment_movie_detail">
28          <argument
29              android:name="movieId"
30              app:argType="integer" />
31      </fragment>
32
33  </navigation>
```

Tabel 16. Source Code Jawaban Soal 1 Modul 3

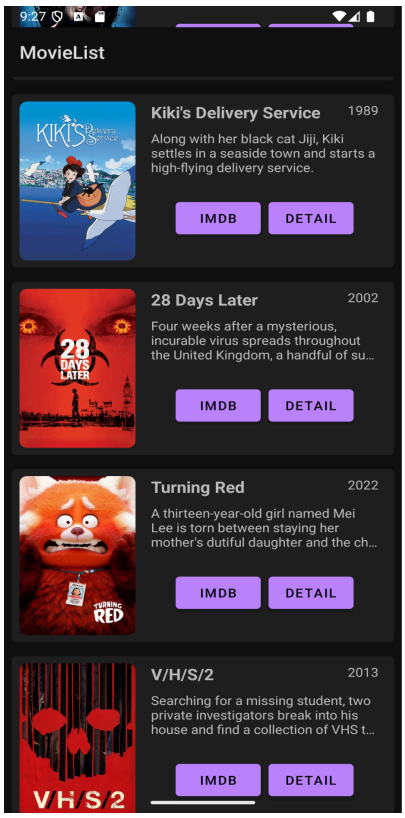
B. Output Program



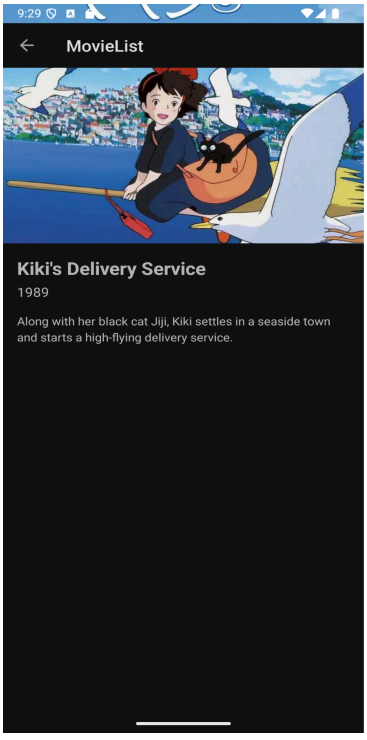
Gambar 13. Screenshot Hasil Jawaban Soal 1 Compose Modul 3



Gambar 14. Screenshot Hasil Jawaban Soal 1 Compose Modul 3



Gambar 15. Screenshot Hasil Jawaban Soal 1 XML Modul 3



Gambar 16. Screenshot Hasil Jawaban Soal 1 XML Modul 3

C. Pembahasan

Jawaban Soal 1
compose

Pada File **MovieListViewModel** Line 68 - 80, itu adalah logik dari intent yang kemudian akan dipakai pada screen atau fragment. (1 D)
Pada File **MovieListScreen** Line 54 - 68, itu adalah penggunaan intent yang akan dijalankan oleh corotine jika fungsi viewmodel dipanggil.(1 D)
Pada File **MovieListScreen** Line 83 - 110, itu adalah penggunaan Lazycolum.(1 A)

XML
Pada File **MovieListAdaptor** Line 26 - 56, itu adalah penggunaan penerapan viewbinding dari RecycleView dimana loop viewbind untuk menapilkan ui setiap item.
Pada File **MovieListFragment** Line 58 - 86, itu adalah penerapan dari intent untuk keluar ke browser dan untuk navigasi ke detail fragment.
Pada File **nav_graph**, itu adalah penggunaan navigasi pada xml dan terdapat 2 fragment yang menjadi destinasi.

Jawaban Soal 2
Ada beberapa kelebihan recycle view dari pada compose salah satunya adalah: fleksibilitas dan kontrol sangat tinggi, performa sangat efisien dan spesifik. Berbeda compose terabstraksi, dan general efisien. Situasi dimana recyle view masih diperlukan: bekerja di proyek lama (legacy codebase), kebutuhan animasi item yang sangat kompleks, dan kebutuhan performa yang sangat spesifik.

D. Tautan Git

Berikut adalah tautan untuk source code yang telah dibuat.
https://github.com/biahlil/Pemrograman-Mobile.git

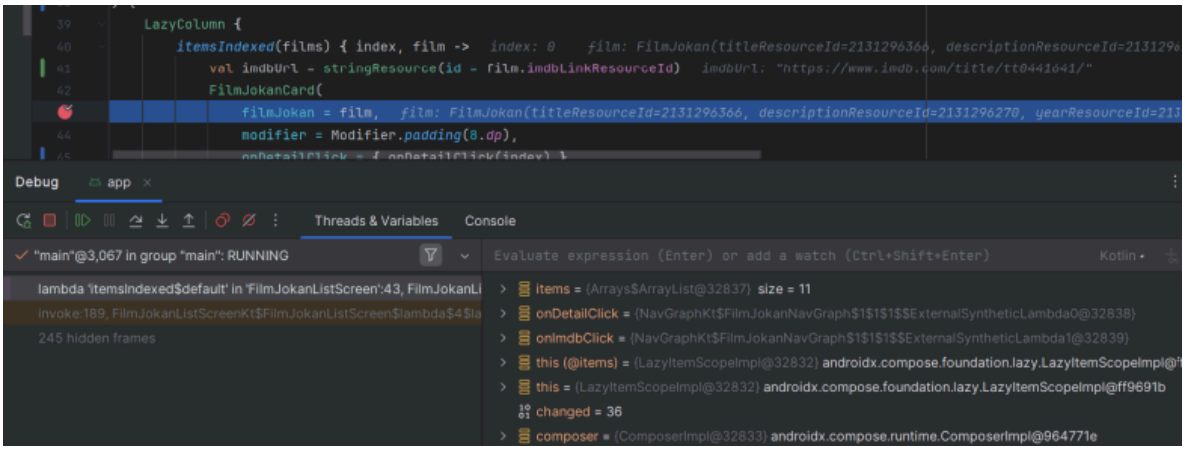
MODUL 4

SOAL

Soal Praktikum:

1. Lanjutkan aplikasi Android berbasis XML dan Jetpack Compose yang sudah dibuat pada Modul 3 dengan menambahkan modifikasi sesuai ketentuan berikut:
    a. Buatlah sebuah ViewModel untuk menyimpan dan mengelola data dari list item. Data tidak boleh disimpan langsung di dalam Fragment atau Activity.
    b. Gunakan ViewModelFactory untuk membuat parameter dengan tipe data String di dalam ViewModel.
    c. Gunakan StateFlow untuk mengelola event onClick dan data list item dari ViewModel ke Fragment.
    d. Install dan gunakan library Timber untuk logging event berikut:
        i. Log saat data item masuk ke dalam list
        ii. Log saat tombol Detail dan tombol Explicit Intent ditekan
        iii. Log data dari list yang dipilih ketika berpindah ke halaman Detail
    e. Gunakan tool Debugger di Android Studio untuk melakukan debugging pada aplikasi. Cari setidaknya satu breakpoint yang relevan dengan aplikasi. Lalu, gunakan fitur Step Into, Step Over, dan Step Out. Setelah itu, jelaskan fungsi Debugger, cara menggunakan Debugger, serta fitur Step Into, Step Over, dan Step Out

2. Jelaskan Application class dalam arsitektur aplikasi Android dan fungsinya. Aplikasi harus dapat mempertahankan fitur-fitur yang sudah dibuat pada modul sebelumnya. Berikut adalah contoh debugging dalam Android Studio.



Gambar 17. Contoh Penggunaan Debugger

A. Source Code

Compose
MovieListViewModel.kt

```
1   package com.example.movielist.ui.movielist
2
3   import androidx.compose.runtime.State
4   import androidx.compose.runtime.mutableStateOf
5   import androidx.lifecycle.ViewModel
6   import androidx.lifecycle.viewModelScope
7   import com.example.movielist.data.Result
8   import com.example.movielist.data.repository.MovieRepository
9   import
10  com.example.movielist.data.repository.impl.FakeMovieReposito
11  ry
12  import com.example.movielist.domain.model.Movie
13  import kotlinx.coroutines.flow.MutableSharedFlow
14  import kotlinx.coroutines.flow.asSharedFlow
15  import kotlinx.coroutines.launch
16  import timber.log.Timber
17
18
19  class MovieListViewModel(
20              val   movieRepository   :   MovieRepository   =
21  FakeMovieRepository()
22  ) : ViewModel() {
23
24      private val _eventFlow = MutableSharedFlow<UiEvent>()
25      val eventFlow = _eventFlow.asSharedFlow()
26
27                      private     val     _movieListState     =
28  mutableStateOf(MovieListState())
29              val   movieListState:   State<MovieListState>   =
30  _movieListState
31
32      init {
33          getAllMovies()
34      }
35
36      fun getAllMovies() {
37          viewModelScope.launch {
38              _movieListState.value = MovieListState(isLoading
39  = true)
40
41              movieRepository.getAllMovies().collect { result
42  ->
43                  when (result) {
44                      is Result.Success<List<Movie>> -> {
45                                          _movieListState.value =
46  MovieListState(movies = result.data, isLoading = false)
47
```

```
48
49  Timber.tag("MovieListViewModel").i("Movies:
50  ${_movieListState.value}")
51
52                      }
53                      is Result.Error -> {
54                                          _movieListState.value =
55  MovieListState(isLoading = false)
56                          _eventFlow.emit(
57                              UiEvent.ShowSnackbar(
58                                                      message =
59  result.exception.message ?: "Unknown Error"
60                              )
61                          )
62                      }
63                  }
64              }
65          }
66      }
67
68      fun onOpenUrlClicked(url: String) {
69          viewModelScope.launch {
70              if (url.isNotBlank()) {
71                  _eventFlow.emit(UiEvent.OpenUrl(url))
72                    Timber.tag("MovieListViewModel").i("intent
73  into URL: $url")
74              } else {
75                  _eventFlow.emit(UiEvent.ShowSnackbar("URL is
76  not valid"))
77              }
78          }
79      }
80  }
81
82  data class MovieListState(
83      val movies: List<Movie> = emptyList(),
84      val isLoading: Boolean = true
85  )
86
87  sealed class UiEvent {
88      data class ShowSnackbar(val message: String) : UiEvent()
89      data class OpenUrl(val url: String) : UiEvent()
90  }
```

Tabel 17. Source Code Jawaban Soal 1 Modul 4

MovieDetailViewModel.kt

```
1   package com.example.movielist.ui.moviedetail
2
3   import androidx.compose.runtime.State
4   import androidx.compose.runtime.mutableStateOf
```

```
5   import androidx.lifecycle.ViewModel
6   import androidx.lifecycle.viewModelScope
7   import com.example.movielist.data.Result
8   import com.example.movielist.data.repository.MovieRepository
9   import
10  com.example.movielist.data.repository.impl.FakeMovieReposito
11  ry
12  import com.example.movielist.domain.model.Movie
13  import com.example.movielist.ui.movielist.UiEvent
14  import kotlinx.coroutines.flow.MutableSharedFlow
15  import kotlinx.coroutines.flow.asSharedFlow
16  import kotlinx.coroutines.launch
17  import timber.log.Timber
18
19
20  class MovieDetailViewModel(
21          private  val  movieRepository:  MovieRepository  =
22  FakeMovieRepository(),
23  ) : ViewModel() {
24                  private    val    _movieDetailState    =
25  mutableStateOf(MovieDetailState())
26          val  movieDetailState:  State<MovieDetailState>  =
27  _movieDetailState
28
29                      private     val     _eventFlow     =
30  MutableSharedFlow<UiEvent.ShowSnackbar>()
31      val eventFlow = _eventFlow.asSharedFlow()
32
33      fun getMovieDetail(id: Int?) {
34          if (id == null) {
35              viewModelScope.launch {
36                  _eventFlow.emit(UiEvent.ShowSnackbar("Movie
37  ID tidak valid."))
38              }
39              return
40          }
41          viewModelScope.launch {
42                                  _movieDetailState.value  =
43  MovieDetailState(isLoading = true)
44
45              movieRepository.getMovie(id).collect { result ->
46                  when (result) {
47                      is Result.Success -> {
48                                      _movieDetailState.value =
49  MovieDetailState(
50                              movie = result.data,
51                              isLoading = false
52                          )
53
54  Timber.tag("MovieDetailViewModel").i("Get into Movie Detail:
55  ${_movieDetailState.value}")
56                      }
```

```
57
58                      is Result.Error -> {
59                                          _movieDetailState.value =
60  MovieDetailState(isLoading = false)
61                          _eventFlow.emit(
62                              UiEvent.ShowSnackbar(
63                                                      message =
64  result.exception.message ?: "Movie not found"
65                              )
66                          )
67                      }
68                  }
69              }
70          }
71      }
72  }
73
74  data class MovieDetailState(
75      val movie: Movie? = null,
76      val isLoading: Boolean = true
77  )
```

Tabel 18. Source Code Jawaban Soal 1 Modul 4

XML
MainActivity.kt

```
1   package com.example.movielistxml
2
3   import androidx.appcompat.app.AppCompatActivity
4   import android.os.Bundle
5   import androidx.navigation.NavController
6   import androidx.navigation.findNavController
7   import androidx.navigation.fragment.NavHostFragment
8   import
9   androidx.navigation.ui.setupActionBarWithNavController
10  import
11  com.example.movielistxml.databinding.ActivityMainBinding
12
13  class MainActivity : AppCompatActivity() {
14
15      private lateinit var binding: ActivityMainBinding
16      private lateinit var navController: NavController
17
18      override fun onCreate(savedInstanceState: Bundle?) {
19          super.onCreate(savedInstanceState)
20                                              binding     =
21  ActivityMainBinding.inflate(layoutInflater)
22          setContentView(binding.root)
23
24          val navHostFragment = supportFragmentManager
25
```

```
                .findFragmentById(R.id.nav_host_fragment) as
NavHostFragment // Use the ID from your XML
        navController = navHostFragment.navController

        setupActionBarWithNavController(navController)
    }

    override fun onSupportNavigateUp(): Boolean {
                                    val    navController    =
findNavController(R.id.nav_host_fragment)
                    return  navController.navigateUp()  ||
super.onSupportNavigateUp()
    }
}
```

Tabel 19. Source Code Jawaban Soal 1 Modul 4

MovieListAdapter.kt

```
package com.example.movielistxml

import android.view.LayoutInflater
import android.view.ViewGroup
import androidx.core.content.ContextCompat.getString
import androidx.recyclerview.widget.DiffUtil
import androidx.recyclerview.widget.ListAdapter
import androidx.recyclerview.widget.RecyclerView
import com.example.movielistxml.databinding.ItemMovieBinding
import com.example.movielistxml.domain.MovieModel

class MovieListAdapter(
    private val onDetailClick: (MovieModel) -> Unit,
    private val onBrowserClick: (MovieModel) -> Unit
)                :                   ListAdapter<MovieModel,
MovieListAdapter.MovieViewHolder>(MovieDiffCallback()) {

       override  fun  onCreateViewHolder(parent:  ViewGroup,
viewType: Int): MovieViewHolder {
                                        val     binding     =
ItemMovieBinding.inflate(LayoutInflater.from(parent.context)
, parent, false)
        return MovieViewHolder(binding)
    }

     override fun onBindViewHolder(holder: MovieViewHolder,
position: Int) {
        val movie = getItem(position)
        holder.bind(movie)
    }

        inner  class  MovieViewHolder(private  val  binding:
ItemMovieBinding) :
        RecyclerView.ViewHolder(binding.root) {
```

```
35          fun bind(movie: MovieModel) {
36              val context = binding.root.context
37                          val  titleText  =  getString(context,
38  movie.titleText)
39                           val  yearText  =  getString(context,
40  movie.yearText)
41                      val descriptionText = getString(context,
42  movie.descriptionText)
43
44
45  binding.ivMoviePoster.setImageResource(movie.image)
46              binding.tvMovieTitle.text = titleText
47              binding.tvMovieYear.text = yearText
48                              binding.tvMovieDescription.text  =
49  descriptionText
50              binding.btnItemDetail.setOnClickListener {
51                  onDetailClick(movie)
52              }
53
54              binding.btnItemBrowser.setOnClickListener {
55                  onBrowserClick(movie)
56              }
57          }
58      }
59  }
60
61  class                    MovieDiffCallback                  :
62  DiffUtil.ItemCallback<MovieModel>() {
63          override  fun  areItemsTheSame(oldItem:  MovieModel,
64  newItem: MovieModel): Boolean {
65          return oldItem.id == newItem.id
66      }
67
68        override  fun  areContentsTheSame(oldItem:  MovieModel,
69  newItem: MovieModel): Boolean {
70          return oldItem == newItem
        }
    }
```

Tabel 20. Source Code Jawaban Soal 1 Modul 4

MovieListViewModel.kt

```
1   package com.example.movielistxml.ui.movielist
2
3   import androidx.lifecycle.LiveData
4   import androidx.lifecycle.MutableLiveData
5   import androidx.lifecycle.ViewModel
6   import androidx.lifecycle.viewModelScope
7   import com.example.movielistxml.data.ResultHandler
8   import
9   com.example.movielistxml.data.repository.MovieRepository
10  import com.example.movielistxml.domain.MovieModel
```

```
11  import kotlinx.coroutines.launch
12
13  class MovieListViewModel(
14      private val movieRepository: MovieRepository
15  ) : ViewModel() {
16
17                          private      val      _movies      =
18  MutableLiveData<List<MovieModel>>()
19      val movies: LiveData<List<MovieModel>> = _movies
20
21      private val _isLoading = MutableLiveData<Boolean>()
22      val isLoading: LiveData<Boolean> = _isLoading
23
24      private val _error = MutableLiveData<String?>()
25
26      init {
27          getAllMovies()
28      }
29
30      private fun getAllMovies() {
31          viewModelScope.launch {
32              _isLoading.value = true
33                      movieRepository.getAllMovies().collect  {
34  movieResult ->
35                  when (movieResult) {
36                      is ResultHandler.Success -> {
37                          _movies.value = movieResult.data
38                      }
39                      is ResultHandler.Error -> {
40                                                  _error.value  =
41  movieResult.exception.message
42                      }
43                  }
44                  _isLoading.value = false
45              }
46          }
47      }
    }
```

Tabel 21. Source Code Jawaban Soal 1 Modul 4

MovieDetailViewModel.kt

```
1   package com.example.movielistxml.ui.moviedetail
2
3   import
4   com.example.movielistxml.data.repository.MovieRepository
5
6   import androidx.lifecycle.LiveData
7   import androidx.lifecycle.MutableLiveData
8   import androidx.lifecycle.ViewModel
9   import androidx.lifecycle.viewModelScope
10  import com.example.movielistxml.data.ResultHandler
```

```
11  import com.example.movielistxml.domain.MovieModel
12  import kotlinx.coroutines.launch
13
14  class MovieDetailViewModel(
15      private val movieRepository: MovieRepository
16  ) : ViewModel() {
17
18      private val _movie = MutableLiveData<MovieModel?>()
19      val movie: LiveData<MovieModel?> = _movie
20
21      private val _isLoading = MutableLiveData<Boolean>()
22      val isLoading: LiveData<Boolean> = _isLoading
23
24      private val _error = MutableLiveData<String?>()
25
26      fun getMovieDetail(movieId: Int) {
27          _isLoading.value = true
28          viewModelScope.launch {
29              _isLoading.value = true
30                   movieRepository.getMovie(movieId).collect {
31  movieResult ->
32                  when (movieResult) {
33                      is ResultHandler.Success -> {
34                          _movie.value = movieResult.data
35                      }
36                      is ResultHandler.Error -> {
37                          _movie.value = null
38                                                  _error.value  =
39  movieResult.exception.message
40                      }
41                  }
42                  _isLoading.value = false
43              }
44          }
45      }
46  }
```

Tabel 22. Source Code Jawaban Soal 1 Modul 4

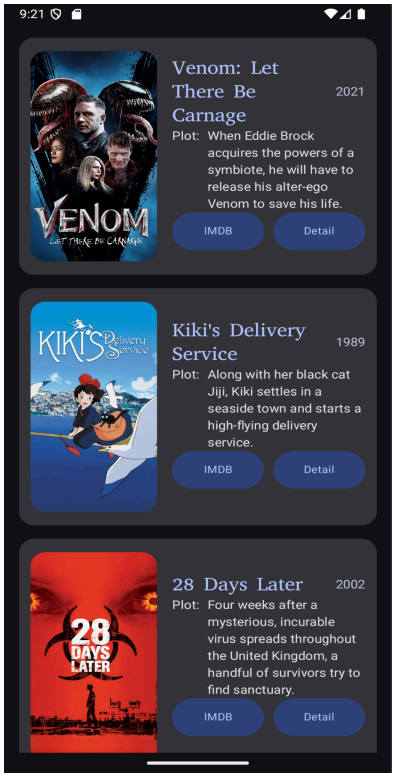
MyMovieApplication.kt

```
1   package com.example.movielistxml
2
3   import android.app.Application
4   import timber.log.Timber
5
6   class MyMovieApplication : Application() {
7       override fun onCreate() {
8           super.onCreate()
9
10          // Inisialisasi Timber.
11           // Ini akan menanam DebugTree, yang secara otomatis
12  akan melakukan logging
```

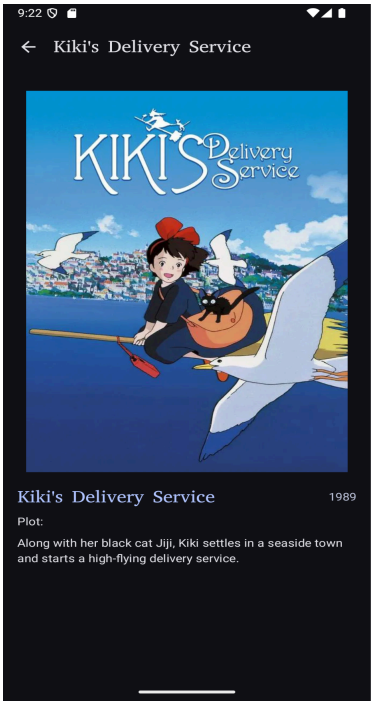
```
13          // dengan tag yang sesuai dan hanya aktif pada build
14  'debug'.
15              // Pada build 'release', tidak ada log yang akan
16  dicetak.
17          Timber.plant(Timber.DebugTree())
18          Timber.i("Timber is initialized.")
19      }
20
21  }
```

Tabel 23. Source Code Jawaban Soal 1 Modul 4

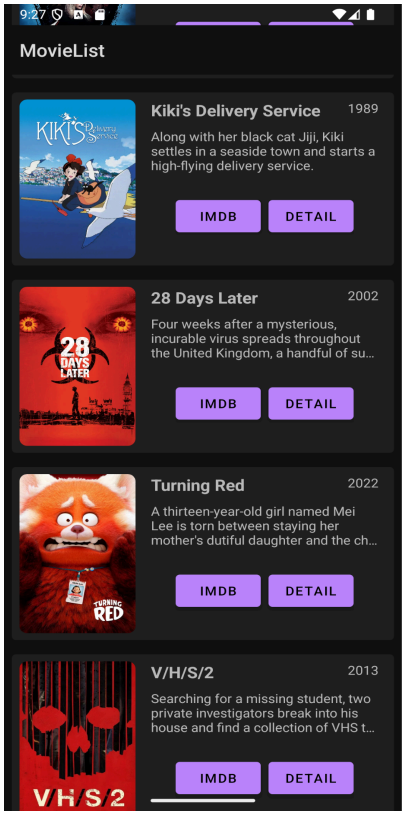
B. Output Program



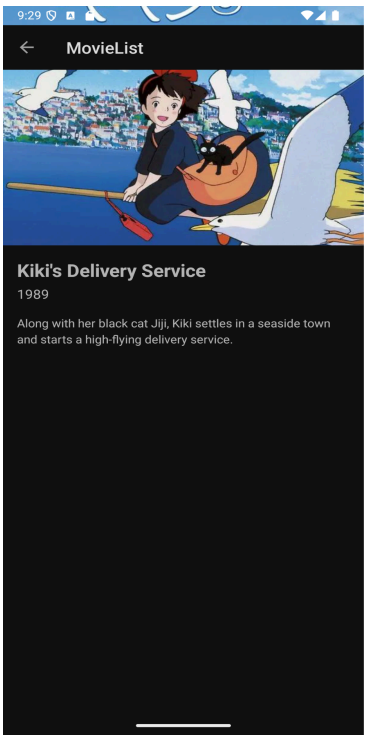
Gambar 18. Screenshot Hasil Jawaban Soal 1 Compose Modul 4



Gambar 19. Screenshot Hasil Jawaban Soal 1 Compose Modul 4



Gambar 20. Screenshot Hasil Jawaban Soal 1 Compose Modul 4



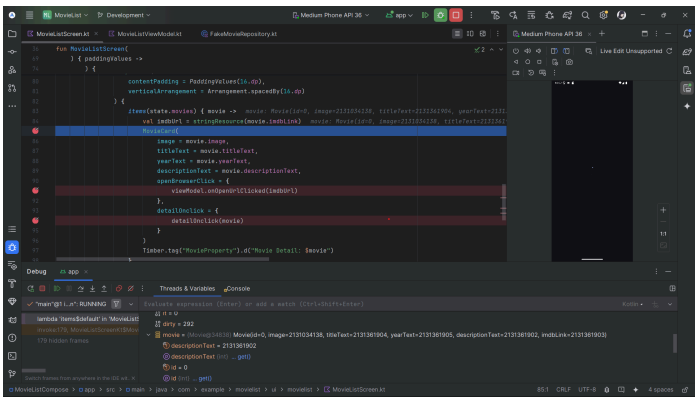
Gambar 21. Screenshot Hasil Jawaban Soal 1 Compose Modul 4

C. Pembahasan

Jawaban Soal 1

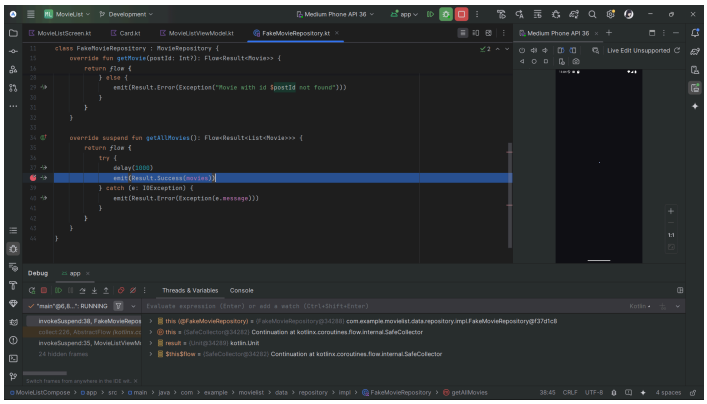
compose
Pada File **MovieListViewModel** Line 36 - 66, itu adalah implementasi dan penggunaan dari StateFlow dimana flow yang digunakan adalah cold flow.
Pada File **MovieDetailViewModel** Line 33 - 70, itu adalah implementasi dan penggunaan dari StateFlow dimana flow yang digunakan adalah cold flow.
Pada File **MovieListViewModel** Line 49 dan 72, itu adalah implementasi dari logging menggunakan timber untuk log moviesstate value dan url pada intent.
Pada File **MovieDetailViewModel** Line 54, itu adalah implementasi dari logging menggunakan timber untuk log data dari moviedetailstate sesuai dengan yang dipilih.

Debugger:
Breakpoint bisanya diletakan pada iterasi item ui, perubahan state ui, feching data di viewmodel, emit flow di repository.



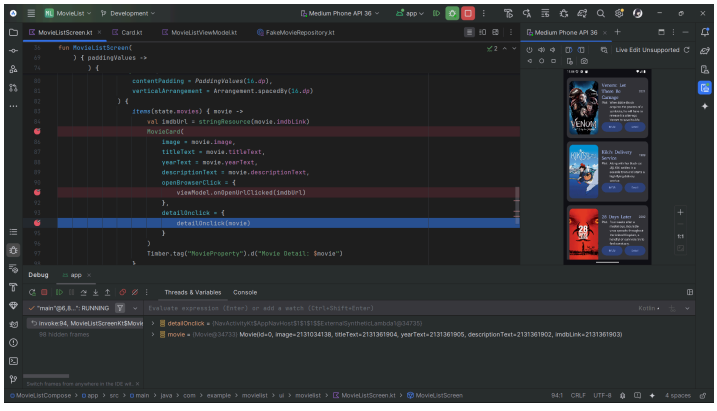
Gambar 22. Screenshot Break point dan debugger

Ini adalah posisi awal dari iterasi item ui, jika kita step out maka akan bisa dilihat bagian kode mana saja yang berjalan diatas iterasi pada saat ini sebelum iterasi selesai. Bisa dilihat fungsi dari repository perlu selesai terlebih dahulu sebelum iterasi item ui dapat selesai.



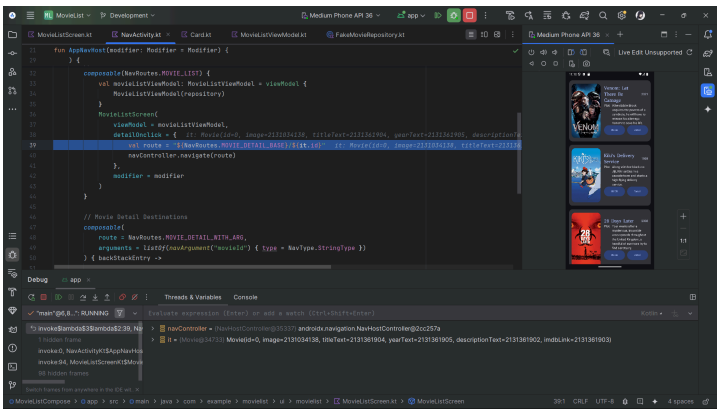
Gambar 23. Screenshot debugger step out

Ini adalah posisi dimana button detail di click kita akan step in untuk melihat kode apa yang perlu dijalankan untuk menyelesaikan proses ini.



Gambar 24. Screenshot debugger onclick

Seperti yang bisa dilihat untuk menjalankan onclick, navigasi perlu selesai terlebih dahulu, kemudian baru bisa berpindah screen.



Gambar 25 Screenshot debugger onclick stepinto

Jawaban Soal 2

Application class adalah sebuah kelas dasar (base class) dalam sebuah aplikasi Android yang digunakan untuk menjaga state atau kondisi global aplikasi. Fungsi utama dari class ini adalah one time inisialisasi seperti library dan plugin. Selain itu class ini juga digunakan untuk melakukan configurasi tingkat aplikasi

D. Tautan Git

Berikut adalah tautan untuk source code yang telah dibuat.
https://github.com/biahlil/Pemrograman-Mobile.git

MODUL 5

SOAL

Soal Praktikum:

1. Lanjutkan aplikasi Android yang sudah dibuat pada Modul 4 dengan menambahkan modifikasi sesuai ketentuan berikut:
    a. Gunakan networking library seperti Retrofit atau Ktor agar aplikasi dapat mengambil data dari remote API. Dalam penggunaan networking library, sertakan generic response untuk status dan error handling pada API dan Flow untuk data stream.
    b. Gunakan KotlinX Serialization sebagai library JSON.
    c. Gunakan library seperti Coil atau Glide untuk image loading.
    d. API yang digunakan pada modul ini adalah The Movie Database (TMDB) API yang menampilkan data film. Berikut link dokumentasi API:

       https://developer.themoviedb.org/docs/getting-started
    e. Implementasikan konsep data persistence (aplikasi menyimpan data walau pengguna keluar dari aplikasi) dengan SharedPreferences untuk menyimpan data ringan (seperti pengaturan aplikasi) dan Room untuk data relasional.
    f. Gunakan caching strategy pada Room. Dibebaskan untuk memilih caching strategy yang sesuai, dan sertakan penjelasan kenapa menggunakan caching strategy tersebut.
    g. Untuk Modul 5, bebas memilih UI yang ingin digunakan, antara berbasis XML atau Jetpack Compose.
2. Aplikasi harus mempertahankan fitur-fitur yang dibuat pada modul sebelumnya.

A. Source Code

Compose

AppModule.kt

```
1  package com.example.movielist.di
2
3  import android.content.Context
4  import androidx.room.Room
5  import com.example.movielist.data.ktor.ApiService
6  import com.example.movielist.data.ktor.ApiServiceImpl
7  import com.example.movielist.data.ktor.KtorClientProvider
8  import com.example.movielist.data.local.MovieDao
9  import com.example.movielist.data.local.MovieDatabase
10 import com.example.movielist.data.repository.MovieRepository
11 import
12 com.example.movielist.data.repository.fake.FakeMovieReposito
   ry
13 import
14 com.example.movielist.data.repository.MovieRepositoryImpl
15 import dagger.Module
16 import dagger.Provides
17 import dagger.hilt.InstallIn
18 import dagger.hilt.android.qualifiers.ApplicationContext
19 import dagger.hilt.components.SingletonComponent
20 import io.ktor.client.HttpClient
21 import javax.inject.Singleton
22
23
24 @Module
25 @InstallIn(SingletonComponent::class)
26 object AppModule {
27
28     @Provides
29     @Singleton
30     fun provideKtorClient(): HttpClient {
31
```

```
32          return KtorClientProvider.httpClient
33      }
34
35      @Provides
36      @Singleton
37      fun provideApiService(client: HttpClient): ApiService {
38          return ApiServiceImpl(client)
39      }
40
41      @Provides
42      @Singleton
43        fun provideMovieDatabase(@ApplicationContext context:
44  Context): MovieDatabase {
45          return Room.databaseBuilder(
46              context,
47              MovieDatabase::class.java,
48              "movie_database"
49          ).fallbackToDestructiveMigration().build()
50      }
51
52      @Provides
53      @Singleton
54      fun provideMovieDao(database: MovieDatabase): MovieDao {
55          return database.movieDao()
56      }
57
58      @Provides
59      @Singleton
60       fun provideMovieRepository(@ApplicationContext context:
61  Context, api: ApiService, dao: MovieDao): MovieRepository {
62          // Untuk menggunakan repository asli (dengan Ktor)
63          return MovieRepositoryImpl(api, dao)
64
65              // Beri komentar pada baris di atas dan hapus
66  komentar di bawah ini
```

```
67  //          return FakeMovieRepository(context = context)
68      }
69  }
```

Tabel 24. Source Code Jawaban Soal 1 Modul 5

KtorClientProvider.kt

```
1   package com.example.movielist.data.ktor
2
3   import io.ktor.client.*
4   import io.ktor.client.engine.cio.*
5   import io.ktor.client.plugins.contentnegotiation.*
6   import io.ktor.client.plugins.defaultRequest
7   import io.ktor.serialization.kotlinx.json.*
8   import io.ktor.util.appendIfNameAbsent
9   import kotlinx.serialization.json.Json
10
11  object KtorClientProvider {
12      val httpClient: HttpClient by lazy {
13          HttpClient(CIO) {
14              install(ContentNegotiation) {
15                  json(
16                      Json {
17                          ignoreUnknownKeys = true
18                      }
19                  )
20              }
21              engine {
22                  requestTimeout = 15_000
23                  endpoint {
24                      connectTimeout = 15_000
25                      socketTimeout = 15_000
26                  }
27              }
28              // Configure default request parameters
29              defaultRequest {
```

```
                url("https://api.themoviedb.org")
                        headers.appendIfNameAbsent("Accept",
"application/json")
                headers.appendIfNameAbsent("Authorization",
"Bearer
eyJhbGciOiJIUzI1NiJ9.eyJhdWQiOiIwZGJkZGQ5MjBlM2FkNDU0M2RkN2Z
lYjNmOWI2YzViMCIsIm5iZiI6MTc0OTAxODExNS44NzYsInN1YiI6IjY4M2Z
lNjAzMDUzNjE5YTdhZGZkYmVlMCIsInNjb3BlcyI6WyJhcGlfcmVhZCJdLCJ
2ZXJzaW9uIjoxfQ.BBXzrwJuk_3W0YFiosG7wZ9uP_Ll5Zi93ZpQBd4M7iA"
)
            }
            // Configure client-wide settings
            expectSuccess = true
            followRedirects = true
        }
    }
}

```

Tabel 25. Source Code Jawaban Soal 1 Modul 5

ApiServiceImpl.kt

```
package com.example.movielist.data.ktor

import com.example.movielist.data.ktor.dto.MovieDetailDto
import
com.example.movielist.data.ktor.dto.MovieListResponseDto

import io.ktor.client.*
import io.ktor.client.call.*
import io.ktor.client.plugins.*
import io.ktor.client.request.*
import io.ktor.http.*
import timber.log.Timber

class ApiServiceImpl(
    private val client: HttpClient
) : ApiService {

```

```
17
18              override   suspend   fun   getDiscoverMovies():
19  MovieListResponseDto {
20          return try {
21
22              client.get("/3/discover/movie") {
23                  url {
24                              parameters.append("include_adult",
25  "false")
26                              parameters.append("include_video",
27  "false")
28                      parameters.append("language", "en-US")
29                                  parameters.append("sort_by",
30  "popularity.desc")
31                              parameters.append("include_video",
32  "false")
33                      parameters.append("with_genres", "16")
34                  }
35                  accept(ContentType.Application.Json)
36                          Timber.tag("ApiService").d("Berhasil
37  mengambil daftar film")
38              }.body()
39
40          } catch (e: Exception) {
41                    Timber.tag("ApiService").e(e, "Error saat
42  mengambil daftar film")
43              MovieListResponseDto(emptyList())
44          }
45      }
46
47
48           override  suspend  fun  getMovieDetail(id:  Int):
49  MovieDetailDto? {
50          return try {
51              client.get("/3/movie/$id") {
                    url {
                        parameters.append("language", "en-US")
```

```
52                  }
53                  accept(ContentType.Application.Json)
54              }.body()
55          } catch (e: ClientRequestException) {
56              // 4xx errors
57                                      Timber.tag("ApiService").e(e,
58  "ClientRequestException")
59              null
60          } catch (e: Exception) {
61                      Timber.tag("ApiService").e(e, "Error saat
62  mengambil detail film")
63              null
64          }
65      }
66  }
67
68
```

Tabel 26. Source Code Jawaban Soal 1 Modul 5

MovieDao.kt

```
1   package com.example.movielist.data.local
2
3   import androidx.room.*
4   import kotlinx.coroutines.flow.Flow
5
6   @Dao
7   interface MovieDao {
8
9       @Query("SELECT * FROM movies WHERE id = :id")
10      fun getMovieById(id: Int): Flow<MovieEntity?>
11
12      @Query("SELECT * FROM movies")
13      fun getAllMovies(): Flow<List<MovieEntity>>
14
15      @Insert(onConflict = OnConflictStrategy.REPLACE)
```

```
16      suspend fun insertAll(movies: List<MovieEntity>)
17
18      @Query("DELETE FROM movies")
19      suspend fun clearAll()
20
21       @Query("UPDATE movies SET imdbId = :imdbId WHERE id =
22  :movieId")
23      suspend fun updateImdbId(movieId: Int, imdbId: String)
24  }
```

Tabel 27. Source Code Jawaban Soal 1 Modul 5

MovieEntity.kt

```
1   package com.example.movielist.data.local
2
3   import androidx.room.Entity
4   import androidx.room.PrimaryKey
5
6   @Entity(tableName = "movies")
7   data class MovieEntity(
8       @PrimaryKey
9       val id: Int,
10      val title: String,
11      val overview: String,
12      val posterPath: String,
13      val releaseDate: String,
14      val imdbId: String? = null
15  )
```

Tabel 28. Source Code Jawaban Soal 1 Modul 5

Mappers.kt

```
1   package com.example.movielist.data
2
3   import com.example.movielist.data.ktor.dto.MovieFromListDto
4   import com.example.movielist.data.local.MovieEntity
```

```
import com.example.movielist.domain.model.Movie

// Mengubah dari Network Model ke Entity (database)
fun MovieFromListDto.toEntity(): MovieEntity {
    return MovieEntity(
        id = this.id,
        title = this.title ?: "Unknown Title",
                overview = this.overview ?: "No overview
available.",
        posterPath = this.posterPath ?: "",
        releaseDate = this.releaseDate ?: "Unknown Date"
    )
}

// Mengubah dari Entity (database) ke Domain Model (untuk
UI)
fun MovieEntity.toDomainMovie(): Movie {
    val imdbUrl = if (this.imdbId?.isNotEmpty() == true) {
        "https://www.imdb.com/title/${this.imdbId}/"
    } else {
        ""
    }

    return Movie(
        id = this.id.toString(),
        title = this.title,
        description = this.overview,
              imageUrl = if (this.posterPath.isNotEmpty())
"https://image.tmdb.org/t/p/original${this.posterPath}" else
"",
         year = this.releaseDate.split("-").firstOrNull() ?:
"N/A",
        imdbURL = imdbUrl
    )
}
```

Tabel 29. Source Code Jawaban Soal 1 Modul 5

MovieRepositoryImpl.kt

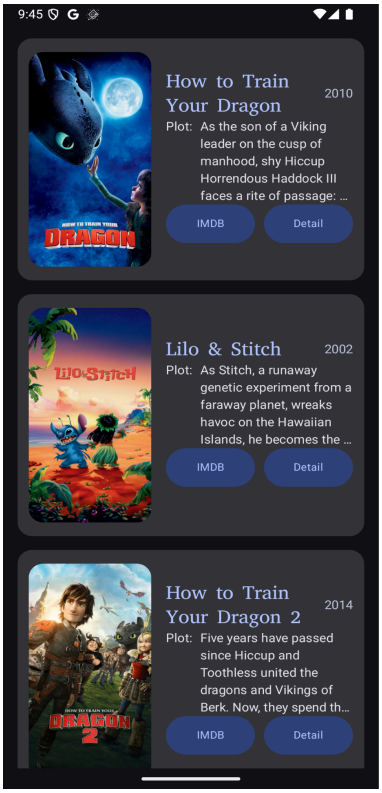
```
1   package com.example.movielist.data.repository
2
3   import com.example.movielist.data.Result
4   import com.example.movielist.data.ktor.ApiService
5   import com.example.movielist.data.local.MovieDao
6   import com.example.movielist.data.toDomainMovie
7   import com.example.movielist.data.toEntity
8   import com.example.movielist.domain.model.Movie
9   import kotlinx.coroutines.coroutineScope
10  import kotlinx.coroutines.flow.Flow
11  import kotlinx.coroutines.flow.channelFlow
12  import kotlinx.coroutines.flow.flow
13  import kotlinx.coroutines.launch
14  import timber.log.Timber
15  import javax.inject.Singleton
16
17  @Singleton
18  class MovieRepositoryImpl (
19      private val api: ApiService,
20      private val dao: MovieDao
21  ) : MovieRepository {
22
23      override fun getAllMovies(): Flow<Result<List<Movie>>> =
24  channelFlow {
25          send(Result.Loading(true))
26
27          try {
28                                          val   remoteMovies   =
    api.getDiscoverMovies().results
29                      val  movieEntities  =  remoteMovies.map  {
30  it.toEntity() }
31
32
```

```
33
34  Timber.Forest.tag("MovieRepositoryImpl").d("getDiscoverMovie
    s: $remoteMovies")
35              dao.clearAll()
36              dao.insertAll(movieEntities)
37
38  Timber.Forest.tag("MovieRepositoryImpl").d("insertAll:
39  $movieEntities")
40
41              coroutineScope {
42                  movieEntities.forEach { movie ->
43                      launch {
44                                                  val  detail  =
    api.getMovieDetail(movie.id)
45
46  Timber.Forest.tag("MovieRepositoryImpl").d("getMovieDetail:
47  $detail")
48                          if (detail?.imdbId != null) {
49                                      dao.updateImdbId(movie.id,
50  detail.imdbId)
51                          }
52                      }
53                  }
54              }
55          } catch (e: Exception) {
56              send(Result.Error(e))
57          } finally {
58              send(Result.Loading(false))
59          }
60          dao.getAllMovies().collect { movies ->
61
62  Timber.Forest.tag("MovieRepositoryImpl").d("getAllMovies:
63  $movies")
64                              send(Result.Success(movies.map  {
    it.toDomainMovie() }))
65          }
66      }
67
```

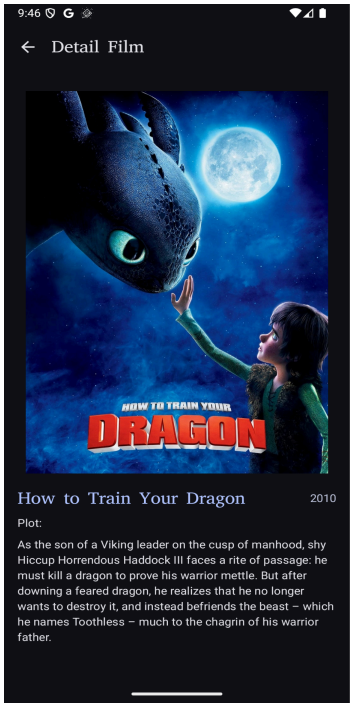
```
    override fun getMovie(postId: Int?): Flow<Result<Movie>>
{
        return flow {
            emit(Result.Loading(true))
            if (postId == null) {
                 emit(Result.Error(Exception("Movie ID tidak
valid.")))
                return@flow
            }
            try {
                dao.getMovieById(postId).collect { entity ->
                    if (entity != null) {

emit(Result.Success(entity.toDomainMovie()))
                    } else {
                            emit(Result.Error(Exception("Film
dengan ID $postId tidak ditemukan di cache.")))
                    }
                }
            } catch (e: Exception) {
                emit(Result.Error(e))
            } finally {
                emit(Result.Loading(false))

            }
        }
    }
}
```

Tabel 30. Source Code Jawaban Soal 1 Modul 5

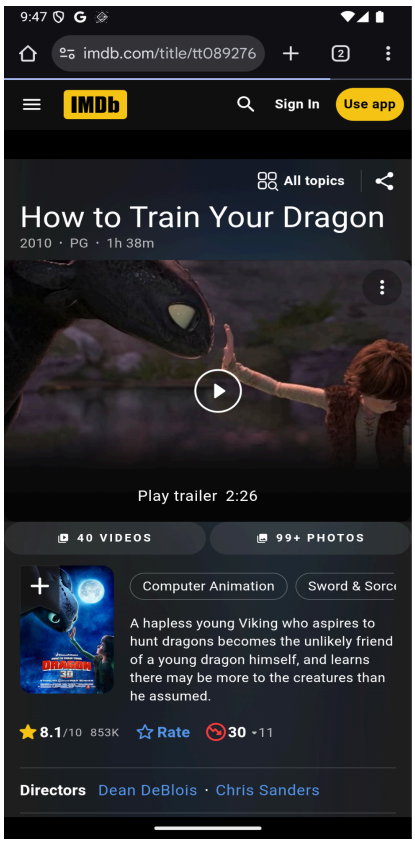
B. Output Program



Gambar 26. Screenshot Hasil Jawaban Soal 1 Compose Modul 5



Gambar 27. Screenshot Hasil Jawaban Soal 1 Compose Modul 5



Gambar 28. Screenshot Hasil Jawaban Soal 1 Compose Modul 5

C. Pembahasan

Compose

AppModule.kt:

Adalah tempat dimana semua dependensi injection provider akan diletakan. Kenapa pakai provider? Karena hampir semuanya menggunakan interface, jadi binding redundant.

KtorClientProvider.kt:

Membuat class client yang berisi config dan default request template yang akan link ke dependensi injection dan bisa digunakan oleh banyak versi dari implementasi api.

ApiServiceImpl.kt:

Tempat membuat request kepada api seperti pada `getDiscoverMovies` dimana melanjutkan config dari client provider.

MovieDao.kt:

Data Access Object adalah interface dimana anda dapat melakukan query kedalam database yang akan dibuat oleh room.

MovieEntity.kt:

Movie Entity merupakan data class model atau table pada database.

Mappers.kt:

Mappers adalah tempat menempatkan global fungsi yang bekerja untuk merubah dan mengatur data dari network ke local ke model ui.

MovieRepositoryImpl.kt:

Tempat dimana implementasi dari apiservice, dan room. Data dari sini akan di teruskan kepada view model untuk dijalankan. Repository akan melakukan emit state.

D. Tautan Git

Berikut adalah tautan untuk source code yang telah dibuat.

https://github.com/biahlil/Pemrograman-Mobile.git